

ชื่อโครงงาน Disney Lorcana PlayLab

โดย

1. นายชยุต	บุญวัฒน์	รหัสนักศึกษา 67070032
2. นายธนัทภัทร	พรหมทอง	รหัสนักศึกษา 67070069
3. นายภูริ	ประชาสุขสิน	รหัสนักศึกษา 67070137
4. นางสาววรรณณิศา	อมรวงศ์ไพบูลย์	รหัสนักศึกษา 67070155
5. นายอสิธารา	พุ่มดอกไม้	รหัสนักศึกษา 67070199
6. นายวรธิษณ์	คงทอง	รหัสนักศึกษา 67070275

เสนอ

ดร. ธนานพ ทองถาวร

ผศ.ดร. พัฒนพงษ์ ฉันทมิตรโอภาส

ผศ.ดร. ลภัส ประดิษฐ์ทัศนีย์

โครงงานนี้เป็นส่วนหนึ่งของวิชาเทคโนโลยีกลุ่มเมฆ

แขนงวิชา การพัฒนาสื่อประสมและเกม

คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ภาคเรียนที่ 1 ปีการศึกษา 2569

คำนำ

เทคโนโลยีกลุ่มเมฆมีบทบาทสำคัญอย่างยิ่งในการพัฒนาระบบสารสนเทศและแอปพลิเคชันยุคใหม่ ด้วยคุณสมบัติเด่นด้านความยืดหยุ่น ความพร้อมใช้งานสูง และความสามารถในการปรับขนาดทรัพยากรได้ตามความต้องการใช้งานจริง โดยไม่ต้องลงทุนในโครงสร้างพื้นฐานฮาร์ดแวร์ล่วงหน้า รายงานฉบับนี้จัดทำขึ้นเพื่อประกอบการเรียนการสอนรายวิชาเทคโนโลยีกลุ่มเมฆ คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

โครงงาน Disney Lorcana PlayLab Cloud มุ่งเน้นการวิเคราะห์ ออกแบบ และพัฒนาระบบจำลองกระดานเล่นการ์ดและวิเคราะห์เด็คแบบเรียลไทม์บนเว็บแอปพลิเคชัน โดยประยุกต์ใช้บริการคลาวด์บน Amazon Web Services (AWS) ออกแบบสถาปัตยกรรม Lean Multi-AZ VPC ร่วมกับ Application Load Balancer, EC2 Auto Scaling Group, Docker Container, และ Amazon DynamoDB เพื่อสร้างระบบที่มีประสิทธิภาพสูง ตอบสนองได้อย่างรวดเร็ว มี Latency ต่ำกว่า 100 มิลลิวินาที และควบคุมค่าใช้จ่ายให้อยู่ภายใต้งบประมาณจำกัด $50 ของ AWS Academy Learner Lab ได้อย่างมีประสิทธิภาพ

คณะผู้จัดทำหวังเป็นอย่างยิ่งว่า ข้อมูล รายละเอียดสถาปัตยกรรม และผลการทดสอบที่นำเสนอในรายงานความก้าวหน้าฉบับนี้ จะเป็นประโยชน์ต่อการศึกษาและเป็นแนวทางในการพัฒนาระบบคลาวด์สำหรับผู้ที่สนใจ หากรายงานฉบับนี้มีข้อบกพร่องหรือข้อผิดพลาดประการใด คณะผู้จัดทำขอน้อมรับคำชี้แนะเพื่อนำไปปรับปรุงในระยะต่อไป

คณะผู้จัดทำ

กันยายน 2569

หัวข้อโครงงาน : Disney Lorcana PlayLab

นักศึกษา : นายชยุต บุญวัฒน์ , นายธนัทภัทร พรหมทอง, นายภูริ ประชาสุขสิน ,
นางสาววรรณณิศา อมรวงศ์ไพบูลย์, นายอสิธารา พุ่มดอกไม้ ,
นายวรธิษณ์ คงทอง

แขนงวิชา : การพัฒนาสื่อประสมและเกม

ปีการศึกษา : 1/2569

บทคัดย่อ

โครงงาน Disney Lorcana PlayLab Cloud นำเสนอการวิเคราะห์ ออกแบบ และพัฒนาระบบจำลองห้องเล่นและวิเคราะห์เด็คการ์ดแบบเรียลไทม์บนสถาปัตยกรรม Hybrid Cloud ที่มี Primary IaaS Stack บน Custom Lean Multi-AZ VPC ครอบคลุม Availability Zone us-east-1a และ us-east-1b, Application Load Balancer, EC2 Auto Scaling Group และ Multi-stage Docker Container ควบคู่กับ Secondary Serverless Stack บน AWS Lambda, Amazon DynamoDB และ AWS API Gateway เพื่อรองรับการเล่นการ์ดสะสม Disney Lorcana TCG ได้อย่างเสถียรและต่อเนื่อง มีความหน่วงในการตอบสนองต่ำกว่า 100 มิลลิวินาที พร้อมกลไก Fault Tolerance และการปรับขนาดอัตโนมัติเมื่อ CPU เฉลี่ยสูงกว่า 60% ภายใต้งบประมาณจำกัด $50 ของ AWS Academy Learner Lab

ในส่วนของส่วนติดต่อผู้ใช้ Frontend พัฒนาด้วย React 19, TypeScript 5.x, Tailwind CSS v4, Framer Motion และ Zustand เพื่อส่งมอบประสบการณ์การเล่นระดับพรีเมียม ประกอบด้วยระบบจำลองฟิสิกส์ 3 มิติ, ระบบตรวจสอบการ์ด 3D Holographic Foil Inspector, การจำลองเปิดซองการ์ด Booster Pack 12 ใบด้วยอัลกอริทึม Fisher-Yates True Randomization, และระบบจัดการเด็คการ์ดที่เชื่อมต่อกับฐานข้อมูลการ์ดอย่างเป็นทางการครบถ้วน 3,242 ใบ ครอบคลุมตั้งแต่ชุด The First Chapter จนถึง Attack of the Vine

การดำเนินงานตามระเบียบวิธี Agile Scrum ได้ลุล่วงตามแผนงาน Sprint 1 ถึง Sprint 3 ครบถ้วน 100% ครอบคลุมการสร้าง Core Gameplay UI, ระบบยืนยันตัวตนด้วย bcrypt และ JWT,

ระบบห้องแข่งขันแบบเรียลไทม์ Sub-100ms Action Sync พร้อมระบบ Rejoin Grace Period 60 วินาที และ Voluntary Exit Match, การย้ายระบบสู่ Nginx Reverse Proxy และ Container บน EC2, ตลอดจนการพัฒนาสคริปต์ 1-Click Scale-to-Zero เพื่อควบคุมต้นทุน Compute ให้เหลือ $0.00 ต่อชั่วโมงเมื่อไม่ได้ใช้งาน และตัดต้นทุน NAT Gateway เหลือ $0.00 ด้วยการใช้ Chained Security Groups ส่งผลให้ระบบมีความพร้อมอย่างสมบูรณ์สำหรับการประเมินความก้าวหน้า Stage 2

คำสำคัญ: Cloud Computing, Multi-AZ VPC, Application Load Balancer, Auto Scaling Group, Docker Container, WebSockets, Disney Lorcana TCG, AWS Learner Lab, Scale-to-Zero

Project Title : Disney Lorcana PlayLab

Students : Mr. Chayut Boonwat, Mr. Thanatphat Promthong,
Mr. Phuri Prachasukhsin, Miss Wannanisa Amornwongphaibun,
Mr. Asithara Phumdorkmai, Mr. Woratit Khongthong

Field of Study : Multimedia and Game Development

Academic Year : 1/2026

Abstract

The Disney Lorcana PlayLab Cloud project presents the architectural design, engineering analysis, and implementation of a real-time multiplayer tabletop simulator and deck analytics platform for the Disney Lorcana Trading Card Game (TCG). The platform is built on a resilient Hybrid Cloud Architecture featuring a Primary IaaS Containerized Stack (Custom Lean Multi-AZ VPC across us-east-1a and us-east-1b, Application Load Balancer, EC2 Auto Scaling Group, and Multi-stage Alpine Docker) alongside a Secondary Serverless Stack (AWS Lambda, Amazon DynamoDB, AWS API Gateway). The system achieves ultra-low synchronization latency (<100ms), automated multi-zone fault tolerance, and dynamic elasticity (Target Tracking CPU Utilization > 60%) within the strict budgetary constraints of the AWS Academy Learner Lab ($50 allocation).

The frontend client is engineered using React 19, TypeScript 5.x, Tailwind CSS v4, Zustand global state orchestration, and Framer Motion, delivering a high-fidelity tabletop simulation connected to an official catalog of 3,242 cards. Core capabilities include real-time mouse tilt physics, 3D holographic foil inspection, and a 12-card Booster Pack simulator driven by the Fisher-Yates randomization algorithm.

Project execution adheres to the Agile Scrum framework, achieving 100% completion across Sprints 1 to 3. Key deliverables include the containerized Nginx reverse proxy on EC2, WebSocket Match Lifecycle v1.5.0 featuring a 60-second Rejoin Grace Period and instant Voluntary Exit Match slot reclamation, chained security group topologies eliminating NAT Gateway costs ($32.40/month to $0.00), and 1-Click Scale-to-Zero automation scripts ensuring zero idle compute expense.

Keywords: Cloud Computing, Multi-AZ VPC, Application Load Balancer, Auto Scaling Group, Docker Containers, WebSockets, Disney Lorcana TCG, AWS Learner Lab, Scale-to-Zero

กิตติกรรมประกาศ

โครงงาน Disney Lorcana PlayLab Cloud สำเร็จลุล่วงตามวัตถุประสงค์ของการส่งมอบงานในระยะที่ 2 ได้ด้วยความอนุเคราะห์ คำแนะนำ และการชี้แนะทางวิชาการอันทรงคุณค่ายิ่งจากคณาจารย์ประจำรายวิชาเทคโนโลยีกลุ่มเมฆ คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง ได้แก่ ดร. ธนานพ ทองถาวร, ผศ.ดร. พัฒนพงษ์ ฉันทมิตรโอภาส, และ ผศ.ดร. ลภัส ประดิษฐ์ทัศนีย์ ที่ได้กรุณาถ่ายทอดองค์ความรู้ด้านสถาปัตยกรรมคลาวด์ การออกแบบระบบเครือข่าย Multi-AZ VPC ความปลอดภัย และการบริหารจัดการทรัพยากรอย่างคุ้มค่า ซึ่งเป็นรากฐานสำคัญในการออกแบบและแก้ปัญหาทางวิศวกรรมซอฟต์แวร์ในโครงงานนี้

คณะผู้จัดทำขอขอบพระคุณเพื่อน ๆ นักศึกษาคณะเทคโนโลยีสารสนเทศ ทุกท่านที่ได้ร่วมทดสอบระบบกระดานและห้องเล่นแบบเรียลไทม์ ให้ข้อคิดเห็นด้านประสบการณ์การใช้งาน และร่วมตรวจสอบข้อผิดพลาดของระบบ รวมถึงขอขอบคุณครอบครัวที่ให้การสนับสนุนและเป็นกำลังใจในการศึกษาค้นคว้าตลอดมา

คณะผู้จัดทำ

กันยายน 2569

สารบัญ

หน้า

คำนำ ..... ก

บทคัดย่อภาษาไทย ..... ข

Abstract ..... ค

กิตติกรรมประกาศ ..... ง

สารบัญ ..... จ

สารบัญรูป ..... ฉ

สารบัญตาราง ..... ช

บทที่ 1 บทนำ 1

1.1 ความเป็นมาและความสำคัญของโครงงาน 1

1.2 วัตถุประสงค์ของโครงงาน 1

1.3 ขอบเขตของระบบ 2

1.4 ประโยชน์ที่คาดว่าจะได้รับ 3

1.5 โครงสร้างของรายงาน 3

บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง 4

2.1 ทฤษฎีที่เกี่ยวข้อง 4

2.2 เทคโนโลยีและเครื่องมือที่ใช้ในการพัฒนา 6

2.3 การบริหารจัดการโครงการด้วยระเบียบวิธี Agile Scrum 7

บทที่ 3 การวิเคราะห์และออกแบบระบบ 9

3.1 ความต้องการของระบบ 9

3.2 แผนภาพ Use Case และ Use Case Specification 10

3.3 สถาปัตยกรรมระบบคลาวด์บน AWS 16
3.4 สถาปัตยกรรมเครือข่าย Lean Multi-AZ VPC และความปลอดภัย 17
3.5 การกระจายโหลดและการปรับขนาดอัตโนมัติ 17
3.6 การออกแบบฐานข้อมูล Amazon DynamoDB 18
3.7 วงจรการสื่อสารแบบเรียลไทม์และการจัดการสถานะเกม 19
3.8 การรับมือข้อผิดพลาดและระบบ Caching 21
3.9 การออกแบบตามกรอบ AWS Well-Architected Framework 22
3.10 การแก้ปัญหาข้อจำกัดบน AWS Learner Lab 22
บทที่ 4 การพัฒนาระบบและผลการดำเนินงาน 24
4.1 การพัฒนาส่วนติดต่อผู้ใช้และ Interactive Playmat 24
4.2 การจัดการโครงสร้างพื้นฐานบน AWS Management Console 33
4.3 การพัฒนา Backend และ Docker Containerization 38
4.4 ผลการทดสอบระบบและการประเมินคุณภาพ 39
บทที่ 5 การประเมินค่าใช้จ่าย แผนงานในอนาคต และสรุปผล 29
5.1 การประเมินค่าใช้จ่ายและการควบคุมงบประมาณ 29
5.2 แผนการพัฒนาสำหรับ Stage 3 30
5.3 สรุปผลการดำเนินงานและบทเรียนที่ได้รับ 30
บรรณานุกรม 32
ภาคผนวก 34
ภาคผนวก ก: สคริปต์ปรับใช้ระบบ Lean Multi-AZ VPC, ALB และ ASG 34
ภาคผนวก ข: สคริปต์ 1-Click Scale-to-Zero สำหรับควบคุมงบประมาณ 35
ภาคผนวก ค: สคริปต์ทดสอบระบบอัตโนมัติ Real-Time WebSocket QA Suite 3

สารบัญรูป

หน้า

รูปที่ 2.1 ตราสัญลักษณ์ Amazon Web Services (AWS) 4
รูปที่ 2.2 ตราสัญลักษณ์ Amazon Elastic Compute Cloud (EC2) 4
รูปที่ 2.3 ตราสัญลักษณ์ Elastic Load Balancing (ALB) 5
รูปที่ 2.4 ตราสัญลักษณ์ Auto Scaling Groups (ASG) 5
รูปที่ 2.5 ตราสัญลักษณ์ Amazon Virtual Private Cloud (VPC) 5
รูปที่ 2.6 ตราสัญลักษณ์ Amazon DynamoDB 6
รูปที่ 2.7 ตราสัญลักษณ์ Amazon Simple Storage Service (S3) 6
รูปที่ 2.8 ตราสัญลักษณ์ Amazon CloudWatch 6
รูปที่ 2.9 ตราสัญลักษณ์ React 19 Frontend UI Library 7
รูปที่ 2.10 ตราสัญลักษณ์ TypeScript Language 7
รูปที่ 2.11 ตราสัญลักษณ์ Tailwind CSS Framework 7
รูปที่ 2.12 ตราสัญลักษณ์ Node.js Runtime Environment 8
รูปที่ 2.13 ตราสัญลักษณ์ Express.js Web Framework 8
รูปที่ 2.14 ตราสัญลักษณ์ Figma UI/UX Design Tool 8
รูปที่ 3.1 แผนภาพ Use Case ของระบบ Disney Lorcana PlayLab Cloud 11
รูปที่ 3.2 ผังภาพสถาปัตยกรรมคลาวด์ Lean Multi-AZ IaaS ร่วมกับ Auto Scaling Group 18
รูปที่ 3.3 ผังภาพวงจรการสื่อสารเรียลไทม์ Real-Time WebSocket Synchronization Flow 22
รูปที่ 3.4 ผังกลยุทธ์ Multi-Tier Fallback และ Local Caching ป้องกัน API ภายนอกขัดข้อง 23
รูปที่ 4.1 หน้าหลักระบบก่อนเข้าสู่ระบบ 49
รูปที่ 4.2 หน้าต่างเข้าสู่ระบบแบบเริ่มต้น 50

รูปที่ 4.3 หน้าต่างลงทะเบียน Username & email..... 50

รูปที่ 4.4 หน้าต่างเข้าสู่ระบบกรอกข้อมูลบัญชีผู้ใช้ io5 และรหัสผ่าน.....51

รูปที่ 4.5 หน้าหลักระบบหลังเข้าสู่ระบบด้วยบัญชี io5 และแสดงสถานะ Cloud Sync.....51

รูปที่ 4.6 หน้ารายการห้องประลองออนไลน์..... 52

รูปที่ 4.7 หน้าต่างสร้างห้องประลองและสร้างรหัสห้อง.....52

รูปที่ 4.8 เลือกสำรับการ์ดสำหรับแข่งขันและกด Auto matching.....53

รูปที่ 4.9 หน้าจัดสำรับการ์ด ภาพรวมคลังการ์ด 3000+ ใบ.....53

รูปที่ 4.10 หน้าดูรายละเอียดการ์ดแต่ละการ์แบบละเอียด.....54

รูปที่ 4.11 หน้าสำหรับดูการ์ดภายใน Deck.....54

รูปที่ 4.12 หน้าแดชบอร์ดวิเคราะห์สถิติสำรับการ์ด (Deck Analytics Dashboard).....55

รูปที่ 4.13 หน้าคู่มือ กฎกติกา และขั้นตอนการเล่น.....55

รูปที่ 4.14 หน้าแดชบอร์ดข้อมูลผู้ใช้และเด็คส่วนตัวของบัญชี io5 User Dashboard.....56

รูปที่ 4.12 หน้ากระดานประลองการ์ดเสมือนจริง.....56

รูปที่ 4.13 หน้าการเลือกเลขคู่และเลขคี่ คนที่ทายถูกสามารถเลือกเริ่มก่อน-หลัง.....57

รูปที่ 4.14 หน้าการ Mulligan phase เลือกการ์ดที่ไม่ต้องการตอนเริ่มเล่นเก็บในกองการ์ด.....57

รูปที่ 4.15 การควบคุมการเล่นและการโต้ตอบบนกระดานประลอง.....58

รูปที่ 4.16 ดูรายละเอียดการ์ดระหว่างเล่นและการเปิดปิดแท็บ Your hand.....58

รูปที่ 4.17 หน้าต่างการแชทแบบ Real time.....59

รูปที่ 4.18 หน้าตอนอีกฝั่งใช้การ์ด lore อัพเดตค่า Ink และลงสถานะการลงการ์ด.....59

รูปที่ 4.19 หน้าตอนลงการ์ดแล้วการ์ดมี Ability พิเศษจะทำงานและขึ้นให้อีกฝ่ายเห็น.....60

รูปที่ 4.20 หน้าตอนการ์ดสามารถเลือกไอคอนโจมตีการ์ดคู่แข่งที่หมุนใช้งาน.....60

รูปที่ 4.21 หน้าตอนการ์ดกดปุ่มเพื่อหมุนเก็บ Lore คะแนนที่ใช้จบเกม.....61

รูปที่ 4.22 หน้าตอนการ์ดกดปุ่มเพื่อหมุนเก็บ Lore แล้วระบบคำนวณแล้วว่ารอบหน้าหมุนครบจะชนะเป็น match point.....61

รูปที่ 4.23 หน้าตอนอีกฝั่งกดย้อนกลับ Turn เพื่อขอเล่นใหม่จะขึ้นให้อีกฝั่งกดอนุญาติ.....62

รูปที่ 4.24 หน้าตอนอีกฝั่งกดออกจากเกมแล้วอีกฝั่งขึ้นนับถอยหลัง 60 วินาที.....62

รูปที่ 4.25 หน้าขึ้นเวลาอีกฝั่งไม่กดปุ่ม Rejoin จะขึ้นให้กดออกจากห้อง.....63

รูปที่ 4.26 หน้าขึ้น Rejoin เพื่อสามารถดเข้าห้องไปเล่นต่อในห้องเดิม.....63

รูปที่ 4.27 หน้า Win เมื่อผู้เล่นได้คะแนน lore ครบ 20 ก่อนอีกฝั่งและสรุปผล.....64

รูปที่ 4.28 หน้า Defeat เมื่อผู้เล่นได้คะแนน lore ครบ 20 ก่อนอีกฝั่งและสรุปผล.....64

รูปที่ 4.29 หน้าต่างบันทึกการปรับปรุงระบบ (Patch Notes Modal).....65

รูปที่ 4.30 แดชบอร์ดตรวจสอบสถานะโครงสร้างพื้นฐานระบบคลาวด์จริง Disney Lorcana PlayLab.67

สารบัญตาราง

หน้า

ตารางที่ 2.1 แผนการดำเนินงานและการส่งมอบงานแต่ละ Sprint.....7

ตารางที่ 2.2 กำหนดการประเมินผลรายวิชา Cloud Technology และสถานะความพร้อม.....8

ตารางที่ 3.1 Use Case Description ระบบสมัครสมาชิกและเข้าสู่ระบบ (UC-01).....11

ตารางที่ 3.2 Use Case Description ระบบจัดการและตรวจสอบเด็คการ์ด (UC-02).....11

ตารางที่ 3.3 Use Case Description ระบบค้นหาคู่เล่นและสร้างห้องแข่งขัน (UC-03).....12

ตารางที่ 3.4 Use Case Description ระบบกระดานจำลองและซิงค์การเล่นเรียลไทม์ (UC-04).....13

ตารางที่ 3.5 Use Case Description ระบบจัดการรอบเล่นและการส่งผ่านเทิร์น (UC-05).....13

ตารางที่ 3.6 Use Case Description ระบบขอแก้มือและย้อนสถานะกระดาน (UC-06).....14

ตารางที่ 3.7 Use Case Description ระบบกู้คืนการเชื่อมต่อ Rejoin Grace Period 60s (UC-07).15

ตารางที่ 3.8 Use Case Description ระบบปรับขนาดคลาวด์และ Scale-to-Zero (UC-08).....15

ตารางที่ 3.9 โครงสร้างตารางข้อมูล Amazon DynamoDB.....18

ตารางที่ 3.10 ตารางสรุปข้อจำกัดบน AWS Learner Lab และแนวทางแก้ไข.....23

ตารางที่ 4.1 ผลการทดสอบคุณภาพระบบโดยรวม (Master QA Test Matrix).....27

ตารางที่ 5.1 การเปรียบเทียบและการประเมินค่าใช้จ่ายบนคลาวด์ตลอดภาคการศึกษา.....29

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญของโครงงาน

เกมการ์ดสะสม Disney Lorcana TCG เป็นหนึ่งในเกมการ์ดที่ได้รับความนิยมอย่างก้าวกระโดดทั่วโลก โดยเป็นการร่วมมือระหว่าง Ravensburger และ The Walt Disney Company ที่รวบรวมตัวละครดิสนีย์มาสร้างสรรค์เป็นกลยุทธ์การแข่งขันเชิงลึก ซึ่งได้รับความสนใจอย่างแพร่หลายจากทั้งผู้เล่นสายแข่งขัน (Competitive Players) และนักสะสม (Collectors) ในทุกภูมิภาค

อย่างไรก็ดี ในบริบทของการฝึกซ้อมและการเข้าถึงเกมการ์ด ผู้เล่นส่วนใหญ่ยังคงเผชิญกับข้อจำกัดหลัก 3 ประการ:

1. ต้นทุนการจัดหาการ์ดจริงและข้อจำกัดทางกายภาพ: การ์ดระดับหายาก (Super Rare, Legendary, Enchanted) มีมูลค่าสูงมากในตลาดรอง และการนัดหมายเล่นจำเป็นต้องมีพื้นที่และเวลาตรงกัน ทำให้ผู้เล่นใหม่และผู้เล่นทั่วไปขาดโอกาสในการทดลองจัดเด็คและทดสอบคอมโบการ์ด
2. ภาระต้นทุนโครงสร้างพื้นฐานและความไม่คุ้มค่าของระบบเดิม: แพลตฟอร์มจำลองเกมการ์ดบนเว็บในอดีตมักพึ่งพา Dedicated Virtual Machine ที่ต้องเปิดทำงานทิ้งไว้ตลอด 24 ชั่วโมง ส่งผลให้เกิดค่าใช้จ่ายสูงแม้ไม่มีผู้ใช้งาน และมักเกิดปัญหาเซิร์ฟเวอร์ล่ม (System Outage) เมื่อมีผู้เล่นหลั่งไหลเข้ามาพร้อมกันจำนวนมาก
3. ปัญหาความหน่วงในการส่งข้อมูลแบบเรียลไทม์ (Synchronization Latency): เว็บแอปพลิเคชันทั่วไปมักสื่อสารด้วย HTTP Polling หรือ Long-Polling แบบดั้งเดิม ซึ่งมี Request Overhead สูงและสร้างความหน่วง (Latency) เกินกว่า 300–500 มิลลิวินาที ไม่ตอบสนองต่อการเล่นการ์ดแบบผลัดตาที่ต้องการเห็นการหมุนการ์ด การวางหมึก และแต้ม Lore ที่เปลี่ยนไปในทันที

เพื่อขจัดปัญหาดังกล่าว คณะผู้จัดทำจึงได้พัฒนาโครงงาน Disney Lorcana PlayLab Cloud โดยออกแบบสถาปัตยกรรม Hybrid Cloud บนบริการ Amazon Web Services (AWS) โดยมี Primary IaaS

Stack บน Custom Lean Multi-AZ VPC, Application Load Balancer, EC2 Auto Scaling Group, และ Multi-stage Docker ควบคู่กับ Secondary Serverless Stack บน AWS Lambda, DynamoDB และ API Gateway เพื่อสร้างระบบจำลองกระดานเล่นการ์ดและวิเคราะห์เด็คที่มีความหน่วงต่ำกว่า 100 มิลลิวินาที สามารถขยายขนาดรองรับผู้เล่นได้อัตโนมัติ และควบคุมค่าใช้จ่ายให้อยู่ภายใต้งบประมาณ $50 ของ AWS Academy Learner Lab ได้อย่างมีประสิทธิภาพสูงสุด

1.2 วัตถุประสงค์ของโครงงาน

1. เพื่อออกแบบและพัฒนาระบบจำลองกระดานซ้อมเล่นการ์ด Disney Lorcana แบบเรียลไทม์บนเว็บเบราว์เซอร์ที่มีความหน่วงต่ำกว่า 100 มิลลิวินาที ผ่านช่องทางสื่อสาร WebSockets (RFC 6455)
2. เพื่อประยุกต์ใช้สถาปัตยกรรมคลาวด์แบบ IaaS และ Containerization บน Multi-AZ VPC, ALB, EC2 Auto Scaling Group และ Docker ในการสร้างระบบที่มีความพร้อมใช้งานสูง (High Availability) และปรับขนาดตามโหลดอัตโนมัติเมื่อ CPU เฉลี่ยเกิน 60%
3. เพื่อพัฒนาระบบจัดการเด็คการ์ดและระบบประมวลผลวิเคราะห์เด็คแบบ Asynchronous ผ่าน Amazon SQS พร้อมฐานข้อมูลการ์ดทางการครบถ้วน 3,242 ใบ
4. เพื่อศึกษาและวางมาตรการด้าน Reliability, Availability, Security, และ Cost Optimization ตามกรอบ AWS Well-Architected Framework
5. เพื่อศึกษาและแก้ไขปัญหาข้อจำกัดทางวิศวกรรมบนสภาพแวดล้อมสิทธิ์จำกัดและงบประมาณ $50 ของ AWS Academy Learner Lab ด้วยกลยุทธ์ Lean Multi-AZ VPC และ 1-Click Scale-to-Zero

1.3 ขอบเขตของระบบ

ระบบครอบคลุมฟังก์ชันการทำงานหลัก 9 ด้าน ดังต่อไปนี้:

- ระบบยืนยันตัวตนและความปลอดภัย (LC-01, LC-02): สมัครสมาชิกและเข้าสู่ระบบด้วย bcrypt (10 Salt Rounds) และ JWT State Management บน Local Storage รองรับทั้งผู้ใช้ทั่วไปและบัญชี Guest/Demo
- ระบบจัดการเด็คการ์ด (LC-03): สร้าง ค้นหา แก้ไข และบันทึกเด็คการ์ดลง Amazon DynamoDB พร้อมระบบตรวจสอบกฎกติกาการจัดเด็ค ได้แก่ ความจุขั้นต่ำ 60 ใบ, ใส่การ์ดซ้ำได้ไม่เกิน 4 ใบ, และจำกัดหมึกไม่เกิน 2 สี
- ระบบจำลองการเปิดซองการ์ด Booster Pack (LC-04): สุ่มเปิดการ์ด 12 ใบต่อซองตามอัตราการออกการ์ดจริงด้วยอัลกอริทึม Fisher-Yates True Randomization
- ระบบตรวจสอบการ์ด 3D Card Inspector (LC-05): แสดงผลการ์ดสามมิติพร้อมเอฟเฟกต์ Holographic Foil Shader และการตอบสนองต่อการเคลื่อนที่ของเมาส์
- ระบบจับคู่และห้องแข่งขัน (LC-06): สร้างห้องแข่งขันด้วยรหัส 6 หลัก ระบบค้นหาห้อง และระบบสลับบทบาทผู้เล่น Player 1, Player 2 และ Spectator
- กระดานจำลองการเล่นการ์ด Interactive Playmat (LC-07): จำลองพื้นที่กระดานมาตรฐาน ประกอบด้วย Play Area, Inkwell, Deck, Discard Pile, Hand และ Lore Counter
- ระบบกู้คืนสถานะการเชื่อมต่อ Rejoin Grace Period 60 วินาที (LC-08): รักษาช่องว่างห้องเล่นไว้ 60 วินาทีเมื่อผู้เล่นหลุดจากการเชื่อมต่อ เพื่อให้สามารถเชื่อมต่อกลับเข้ามาเล่นต่อได้โดยเกมไม่หยุดชะงัก
- ระบบออกจากเกม Voluntary Exit Match (LC-09): ส่ง Action LEAVE_ROOM เคลียร์สล็อตในห้องเล่นบน DynamoDB ทันที เพื่อคืนทรัพยากรห้องเล่นให้ระบบ

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ผู้เล่นและคอมมูนิตี้เกมการ์ด Disney Lorcana มีแพลตฟอร์มบนเว็บที่สามารถฝึกซ้อม จัดเด็ค และทดสอบกลยุทธ์ได้อย่างสะดวกรวดเร็วโดยไม่มีค่าใช้จ่าย

2. ได้รับองค์ความรู้และทักษะเชิงลึกในการออกแบบสถาปัตยกรรมคลาวด์แบบ IaaS และ Containerization ที่รองรับการใช้งานจริงระดับ Production

3. ได้แนวทางการแก้ไขปัญหาข้อจำกัดด้านสิทธิ์และงบประมาณบน AWS Learner Lab ที่เป็นประโยชน์ต่อการศึกษาและการต่อยอดโครงการในอนาคต

4. มีระบบต้นแบบในการจัดการ State การเล่นเกมแบบเรียลไทม์ความเร็วสูงผ่าน WebSockets และการกู้คืนสถานะด้วย NoSQL บน Amazon DynamoDB

1.5 โครงสร้างของรายงาน

รายงานฉบับนี้แบ่งเนื้อหาออกเป็น 5 บท ประกอบด้วย:

- บทที่ 1 บทนำ: กล่าวถึงความเป็นมาและความสำคัญของปัญหา วัตถุประสงค์ ขอบเขตของระบบ และประโยชน์ที่คาดว่าจะได้รับ
- บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง: ทฤษฎีคลาวด์ เครือข่าย โพรโทคอล WebSockets ฐานข้อมูล NoSQL กติกาเกม ตลอดจนระเบียบวิธี Agile Scrum และ Tech Stack
- บทที่ 3 การวิเคราะห์และออกแบบระบบ: ข้อกำหนดเชิงฟังก์ชัน แผนภาพ Use Case Diagram, ตาราง Use Case Description 8 ระบบ, สถาปัตยกรรมคลาวด์ Lean Multi-AZ VPC, ALB & ASG, DynamoDB Schemas, Match Lifecycle, Caching, และการรับมือข้อจำกัด AWS Learner Lab
- บทที่ 4 การพัฒนาระบบและผลการดำเนินงาน: รายละเอียดการพัฒนา Frontend React 19, Multi-stage Docker Nginx Containerization, และ Master QA Test Matrix ครอบคลุม Unit Test, E2E, Load Stress Test, และ Real-time WebSocket Automated Test
- บทที่ 5 การประเมินค่าใช้จ่าย แผนงานในอนาคต และสรุปผล: สรุปค่าใช้จ่ายจริงเทียบกับงบประมาณ $50, แผนงาน Sprint 4 ถึง 7 และบทเรียนทางวิศวกรรมที่ได้รับ

บทที่ 2

ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

2.1 ทฤษฎีที่เกี่ยวข้อง

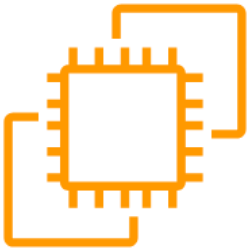
ในการวิเคราะห์และพัฒนาระบบ Disney Lorcana PlayLab Cloud คณะผู้จัดทำได้นำทฤษฎีทางด้านวิทยาการคอมพิวเตอร์ วิศวกรรมคลาวด์ และวิศวกรรมซอฟต์แวร์มาประยุกต์ใช้เป็นรากฐานสำคัญ ประกอบด้วย 7 มิติดังนี้:

2.1.1 สถาปัตยกรรมการประมวลผลแบบกลุ่มเมฆ

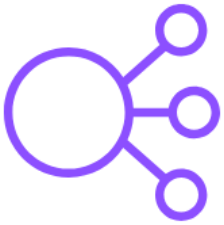
การประมวลผลแบบกลุ่มเมฆแบ่งรูปแบบบริการออกเป็น 3 ระดับหลัก ได้แก่ Infrastructure as a Service (IaaS), Platform as a Service (PaaS), และ Software as a Service (SaaS) โครงงานนี้เลือกใช้โมเดล Hybrid Cloud โดยมี Primary IaaS Stack บน Amazon EC2 ภายใต้ Auto Scaling Group ร่วมกับ Secondary Serverless Stack บน AWS Lambda และ DynamoDB เพื่อสร้างความยืดหยุ่นในการขยายตัว (Elasticity) และความพร้อมใช้งานสูง (High Availability) ข้ามหลาย Availability Zones (Multi-AZ) ช่วยป้องกัน Single Point of Failure (SPOF)



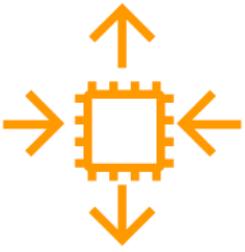
รูปที่ 2.1: ตราสัญลักษณ์ Amazon Web Services (AWS)



รูปที่ 2.2: ตราสัญลักษณ์ Amazon Elastic Compute Cloud (EC2)



รูปที่ 2.3: ตราสัญลักษณ์ Elastic Load Balancing (ALB)



รูปที่ 2.4: ตราสัญลักษณ์ Auto Scaling Groups (ASG)



รูปที่ 2.5: ตราสัญลักษณ์ Amazon Virtual Private Cloud (VPC)



รูปที่ 2.6: ตราสัญลักษณ์ Amazon DynamoDB



รูปที่ 2.7: ตราสัญลักษณ์ Amazon Simple Storage Service (S3)



รูปที่ 2.8: ตราสัญลักษณ์ Amazon CloudWatch

2.1.2 เครือข่ายเสมือนและความปลอดภัยบนคลาวด์

ทฤษฎีเครือข่ายคอมพิวเตอร์ด้านการแบ่งส่วนเครือข่าย (Network Segmentation) ด้วย Classless Inter-Domain Routing (CIDR) ถูกนำมาใช้กำหนด Address Space 10.0.0.0/16 แบ่งเป็น Subnet ประจำโซน us-east-1a (10.0.1.0/24) และ us-east-1b (10.0.2.0/24) สำหรับด้านความปลอดภัย แทนที่จะใช้ NAT Gateway ซึ่งมีค่าใช้จ่ายคงที่สูง ($32.40 ต่อเดือน) ระบบได้นำทฤษฎี Chained Security Groups มาใช้ โดยตั้งค่า Security Group ของเครื่อง EC2 ให้อนุญาตรับทราฟฟิกเฉพาะที่ส่งต่อมาจาก Security Group ของ Application Load Balancer เท่านั้น ทำให้ปลอดภัยจากการสแกนพอร์ตภายนอกเทียบเท่าการวางใน Private Subnet โดยมีต้นทุน $0.00

2.1.3 การกระจายโหลดและความพร้อมใช้งานสูง (Load Balancing & Session Stickiness)

Application Load Balancer (ALB) ทำงานที่ Layer 7 (Application Layer) ตามแบบจำลอง OSI Model ตรวจสอบคำขอ HTTP/HTTPS และเส้นทาง URL เพื่อกระจายโหลดไปยัง Target Group ข้าม AZ พร้อมตรวจสอบสุขภาพ (Health Check) อัตโนมัติ ที่สำคัญคือการเปิดใช้งาน Sticky Sessions ด้วยคุกกี้ lb_cookie ทำให้การเชื่อมต่อแบบ Stateful WebSockets ของผู้เล่นถูกจับคู่ไว้กับ EC2 Instance เครื่องเดิมตลอดการแข่งขัน แม้ระบบ Auto Scaling Group จะสปอว์นเครื่องใหม่เข้ามาในคลัสเตอร์ก็ตาม

2.1.4 โพรโทคอลการสื่อสารสองทิศทางแบบเรียลไทม์

การจำลองกระดานแข่งขันการ์ดต้องการการตอบสนองแบบทันทีทันใด (Sub-100ms Latency) การเปรียบเทียบเชิงเทคนิคแสดงให้เห็นว่า:

- HTTP Short Polling: ส่งคำขอซ้ำทุกช่วงเวลา ก่อให้เกิด Header Overhead มหาศาลและสร้างภาระหนักแก่เซิร์ฟเวอร์
- Server-Sent Events (SSE): เป็นการสื่อสารทางเดียวจากเซิร์ฟเวอร์สู่ไคลเอนต์ ไม่รองรับการส่งคำสั่งเล่นการ์ดแบบสองทิศทาง
- WebSockets (RFC 6455): ทำ Handshake เพียงครั้งเดียวผ่าน HTTP Upgrade จากนั้นสลับเป็น Full-Duplex TCP Connection ที่มี Frame Overhead เพียง 2–10 ไบต์ ช่วยลดปริมาณข้อมูลเครือข่ายลงได้กว่า 85% และทำให้ความหน่วงในการซิงค์กระดานต่ำกว่า 50 มิลลิวินาที

2.1.5 การจัดการสถานะและความสอดคล้องแบบกระจาย

ในระบบที่มีการส่งข้อมูลผ่านเครือข่าย ความหน่วงของเครือข่ายอาจทำให้ผู้เล่นรู้สึกสะดุด ระบบจึงนำทฤษฎี Optimistic UI Updates มาใช้ โดยเมื่อผู้เล่นขยับการ์ดหรือปรับแต้ม Lore หน้าจอไคลเอนต์จะเรนเดอร์การเปลี่ยนแปลงทันทีใน 0ms ขณะเดียวกันจะส่ง Action Payload ผ่าน WebSockets ไปยังเซิร์ฟเวอร์เพื่อกระจายให้คู่แข่ง และบันทึก Snapshot ลง Amazon DynamoDB หากเกิดข้อผิดพลาดจะใช้กลไก State Rollback ย้อนคืนสถานะเดิม

2.1.6 สถาปัตยกรรมฐานข้อมูลแบบ NoSQL

ฐานข้อมูลแบบ NoSQL Key-Value / Document Store บน Amazon DynamoDB ตอบโจทย์ระบบเกมด้วยความสามารถในการอ่านเขียนแบบ Single-digit Millisecond Latency โดยใช้การแฮชค่า Partition Key (PK) และ Sort Key (SK) เพื่อกระจายข้อมูลลงบนโฮสต์ และเปิดใช้กลไก Time-To-Live (TTL) บนค่า Unix Epoch Timestamp เพื่อลบห้องเล่นที่ไม่มีความเคลื่อนไหวทิ้งอัตโนมัติ โดยไม่เสียค่าประมวลผล

2.1.7 ทฤษฎีและโมเดลกติกาเกมสะสมการ์ด Disney Lorcana

Disney Lorcana TCG มีกฎกติกาที่ต้องจำลองเป็น State Machine อย่างแม่นยำ:

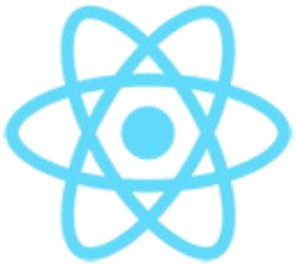
- โมเดลทรัพยากรหมึก (Inkwell Curve): การ์ดที่มีขอบหมึกสามารถคว่ำลงในบ่อหมึกได้เทิร์นละ 1 ใบ เพื่อเปลี่ยนเป็นทรัพยากรจ่ายค่าร่ายการ์ด
- สถานะการ์ด Ready และ Exert: การ์ดที่พร้อมใช้งานจะอยู่ในแนวตั้ง (Ready) เมื่อทำเควสต์หรือโจมตีจะต้องหมุนแนวนอน 90 องศา (Exert) และจะ Ready ใหม่เมื่อเริ่มเทิร์น
- เงื่อนไขการชนะ (Lore Race): ผู้เล่นสะสมแต้ม Lore จากการทำเควสต์ ฝ่ายใดสะสมครบ 20 แต้มก่อนจะเป็นผู้ชนะทันที
- กฎการจัดเด็ค: เด็คต้องมีขนาดอย่างน้อย 60 ใบ, ใช้หมึกได้ไม่เกิน 2 สีจาก 6 สี (Amber, Amethyst, Emerald, Ruby, Sapphire, Steel), และใส่การ์ดชื่อซ้ำกันได้ไม่เกิน 4 ใบ

2.2 เทคโนโลยีและเครื่องมือที่ใช้ในการพัฒนา

ระบบ Disney Lorcana PlayLab Cloud เลือกใช้ชุดเครื่องมือมาตรฐานระดับ Production ที่ทำงานร่วมกันอย่างมีประสิทธิภาพ:

- Frontend: React 19, TypeScript 5.x, Vite 6.x, Tailwind CSS v4, Framer Motion, Lucide React, และ Zustand State Management สำหรับควบคุมกระดาน
- Backend & Realtime: Node.js 20 LTS, Express, WebSockets (RFC 6455 ws), bcryptjs สำหรับแฮชรหัสผ่าน, jsonwebtoken สำหรับ Session Stateless, และ AWS SDK v3
- Cloud Infrastructure: Amazon EC2 (t3.micro), Application Load Balancer (ALB), EC2 Auto Scaling Group (ASG), Amazon Virtual Private Cloud (VPC), Amazon DynamoDB, Amazon SQS, AWS CloudWatch, และ AWS Systems Manager (SSM)
- Containerization & Web Server: Docker Engine, Multi-stage Alpine Build (ลดขนาดเหลือ <90MB), และ Nginx Reverse Proxy (Gzip Compression, WebSocket Upgrading)

• Testing & Automation: Vitest 4.x (Unit & Store Test), Playwright (E2E Browser Automation), Artillery (Load Stress Testing), และ PowerShell Automation Suite บน Windows



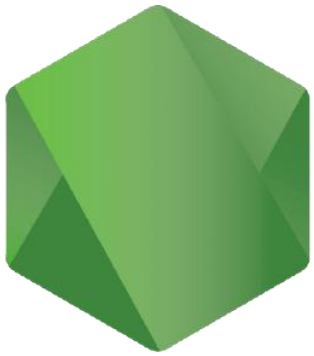
รูปที่ 2.9: ตราสัญลักษณ์ React 19 Frontend UI Library



รูปที่ 2.10: ตราสัญลักษณ์ TypeScript Language



รูปที่ 2.11: ตราสัญลักษณ์ Tailwind CSS Framework



รูปที่ 2.12: ตราสัญลักษณ์ Node.js Runtime Environment



รูปที่ 2.13: ตราสัญลักษณ์ Express.js Web Framework



รูปที่ 2.14: ตราสัญลักษณ์ Figma UI/UX Design Tool

2.3 การบริหารจัดการโครงการด้วยระเบียบวิธี Agile Scrum

โครงการ Disney Lorcana PlayLab Cloud ใช้ระเบียบวิธีปฏิบัติงานแบบ Agile Scrum โดยแบ่งระยะเวลาการพัฒนาออกเป็น 7 Sprints (Sprint ละ 2 สัปดาห์) การดำเนินงานใน Phase ที่ 2 ครอบคลุมการพัฒนาตั้งแต่ Sprint 1 ถึง Sprint 3 ซึ่งมุ่งเน้นการสร้างสถาปัตยกรรมหลักของระบบและเชื่อมต่อฟังก์ชันพื้นฐานครบ 100% ดังแสดงในตารางที่ 2.1

ตารางที่ 2.1: แผนการดำเนินงานและการส่งมอบงานแต่ละ Sprint

Sprint	ช่วงเวลา	เป้าหมายและผลงานที่ส่งมอบ	สถานะ
Sprint 1	1–14 ส.ค. 2569	ตั้งค่าโครงสร้างโปรเจกต์ React 19 + TypeScript, ติดตั้ง Tailwind CSS v4, ออกแบบ UI Theme และระบบจัดการเด็ค	เสร็จสมบูรณ์ 100%
Sprint 2	15–28 ส.ค. 2569	พัฒนาระบบ Booster Pack Gacha, 3D Foil Inspector, เชื่อมต่อ Lorcast API 3,242 ใบ, ทดสอบ Unit Test บน Vitest	เสร็จสมบูรณ์ 100%
Sprint 3	29 ส.ค. – 4 ก.ย. 2569	พัฒนา WebSocket Lifecycle v1.5.0, ย้ายระบบสู่ Multi-AZ VPC + ALB + ASG บน Docker EC2, จัดทำรายงาน Stage 2	เสร็จสมบูรณ์ 100%

Sprint 4	5–25 ก.ย. 2569	พัฒนาระบบ Asynchronous Deck Analytics ผ่าน Amazon SQS และอัลกอริทึมแนะนำการ์ด	แผนงานถัดไป
Sprint 5	26 ก.ย. – 10 ต.ค. 2569	ติดตั้ง AWS X-Ray, CloudWatch Dashboard, ปรับปรุงระบบความปลอดภัยและการจัดการข้อผิดพลาด	แผนงานถัดไป
Sprint 6	11–20 ต.ค. 2569	ทำการทดสอบ Load Testing ขนาดใหญ่ด้วย Artillery และซ้อมการนำเสนอระบบ	แผนงานถัดไป
Sprint 7	21–25 ต.ค. 2569	จัดทำรายงานฉบับสมบูรณ์ บันทึกวิดีโอสาธิตระบบ และเตรียมการนำเสนอครั้งสุดท้าย	แผนงานถัดไป

สรุปผลความก้าวหน้าของโครงการ ณ ปัจจุบันบรรลุตามเป้าหมายของ Stage 2 ครบถ้วน 100% ทุกมิติ ทีมงานได้พัฒนาและทดสอบ Frontend Playmat, ระบบห้องแข่งขันเรียลไทม์, โครงสร้าง Multi-AZ VPC, และระบบ Scale-to-Zero พร้อมสำหรับการประเมินผล ตารางที่ 2.2 แสดงกำหนดการประเมินผลในรายวิชาและความพร้อมในการส่งมอบของกลุ่ม G21:

ตารางที่ 2.2: กำหนดการประเมินผลรายวิชา Cloud Technology และสถานะความพร้อม

กำหนดเวลา	หมวดหมู่การประเมินผล	สถานะความพร้อมของกลุ่ม G21
Stage 1 (สัปดาห์ที่ 4)	นำเสนอหัวข้อและสถาปัตยกรรมเบื้องต้น (ส่งบทที่ 1-3)	ผ่านการประเมินเรียบร้อย
Stage 2 (7-11 ก.ย. 2569)	ส่งมอบความก้าวหน้า นำเสนอและส่งรายงาน (ส่งบทที่ 2-3)	พร้อมส่งมอบ 100% (เอกสารรายงาน + ระบบจริง)
Stage 3 (10 ต.ค. 2569)	ส่งมอบระบบวิเคราะห์ Asynchronous & Monitoring	อยู่ระหว่างเตรียมการพัฒนา Sprint 4
Final Demo (20-25 ต.ค.)	การสาธิตระบบจริงและการนำเสนอผลงาน	พร้อมทดสอบซ้อมระบบ
Final Report	รายงานเล่มสมบูรณ์ฉบับวิชาการ	ร่างเนื้อหาเสร็จสิ้นกว่า 60%
Video Presentation	วิดีโอสาธิตการใช้งานและสถาปัตยกรรม	เตรียมบันทึกภาพหน้าจอและเสียงบรรยาย

บทที่ 3

การวิเคราะห์และออกแบบระบบ

3.1 ความต้องการของระบบ

การวิเคราะห์ความต้องการของระบบ Disney Lorcana PlayLab Cloud แบ่งออกเป็น 2 ส่วนหลัก ตามหลักวิศวกรรมซอฟต์แวร์:

3.1.1 ข้อกำหนดเชิงฟังก์ชัน (Functional Requirements)

- FR-01: ผู้ใช้สามารถลงทะเบียน เข้าสู่ระบบ และจัดการข้อมูลโปรไฟล์ผ่านระบบยืนยันตัวตนด้วย bcrypt และ JWT
- FR-02: ผู้ใช้สามารถค้นหา กรอง และจัดเด็คการ์ดตามกฎกติกาทางการ พร้อมบันทึกลง Amazon DynamoDB
- FR-03: ผู้ใช้สามารถสร้างห้องแข่งขันด้วยรหัส 6 หลัก หรือค้นหาคู่ต่อสู้ในระบบ Matchmaking Queue
- FR-04: ผู้เล่นสามารถขยับการ์ด ปรับแต้ม Lore คว่ำการ์ดลง Inkwell และสั่ง Ready/Exert ผ่าน WebSockets ได้แบบเรียลไทม์
- FR-05: ผู้เล่นสามารถสั่งขอแก้มือ (Undo Action) โดยหักโควตา 1 ครั้ง และเมื่อคู่แข่งอนุมัติ ระบบจะย้อนสถานะกระดานสู่ Snapshot ก่อนหน้าทันที
- FR-06: ผู้ใช้สามารถสุ่มเปิดซองการ์ด Booster Pack 12 ใบ พร้อมตรวจสอบการ์ดแบบสามมิติ (3D Holographic Foil Inspector)

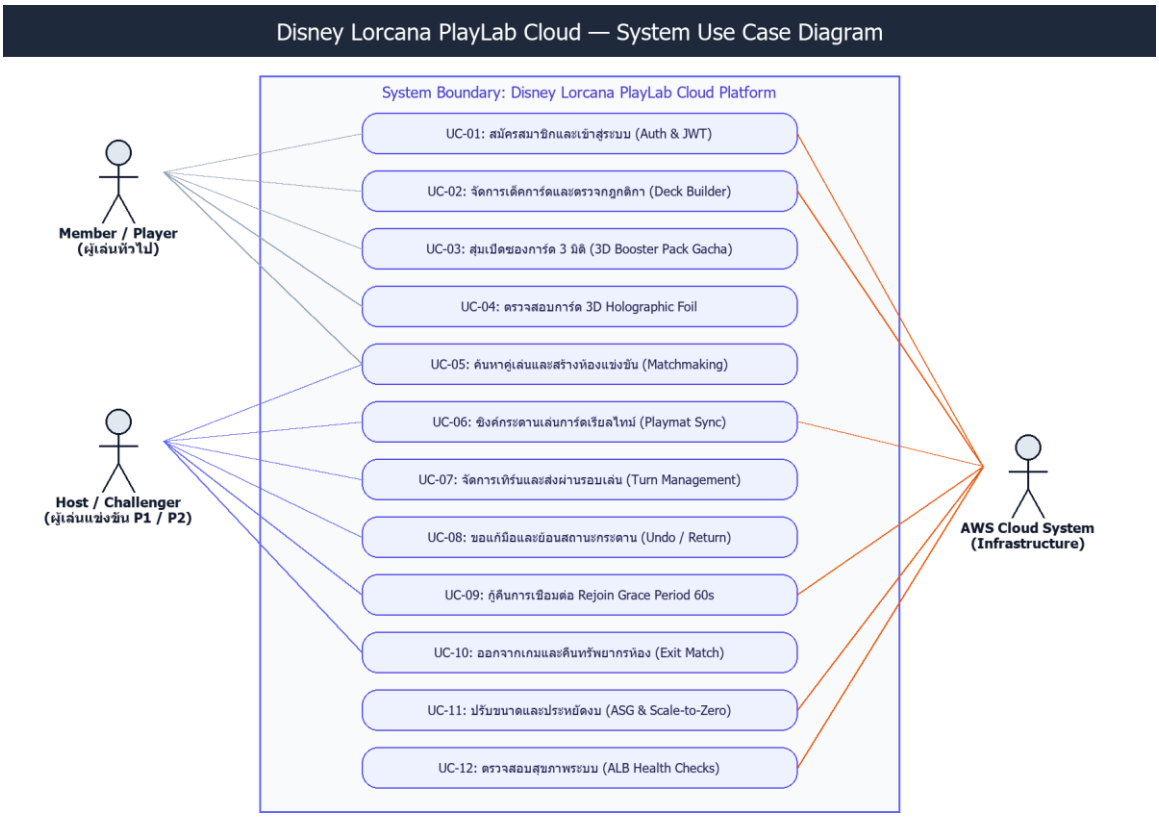
3.1.2 ข้อกำหนดที่ไม่ใช่เชิงฟังก์ชัน (Non-Functional Requirements)

- NFR-01 (Latency): ความหน่วงในการซิงค์ข้อมูลกระดานระหว่างผู้เล่นสองฝั่งต้องต่ำกว่า 100 มิลลิวินาที (เป้าหมายจริง 40–80ms)

- NFR-02 (High Availability & Multi-AZ): ระบบต้องทำงานแบบ Multi-AZ สามารถสลับการทำงานอัตโนมัติหากโซนใดโซนหนึ่งขัดข้อง
- NFR-03 (Scalability): ระบบ Auto Scaling Group ต้องสามารถสปอว์นอินสแตนซ์ EC2 เพิ่มอัตโนมัติเมื่อค่าเฉลี่ย CPU เกิน 60%
- NFR-04 (Cost Constraint): ค่าใช้จ่ายการทำงานของระบบทั้งหมดต้องอยู่ภายใต้งบประมาณจำกัด $50 ของ AWS Academy Learner Lab
- NFR-05 (Security): รหัสผ่านต้องถูกเข้ารหัสด้วย bcrypt 10 rounds, ปิดกั้นพอร์ต EC2 สู่สาธารณะด้วย Chained Security Groups, และเข้าถึงสิทธิ์ผ่าน IMDSv2 โดยไม่ฝัง AWS Credentials ในซอร์สโค้ด

3.2 แผนภาพ Use Case และ Use Case Specification

จากข้อกำหนดความต้องการข้างต้น ระบบได้รับการจำแนกปฏิสัมพันธ์ระหว่างผู้ใช้งานและระบบคลาวด์ ดังแสดงในแผนภาพ Use Case Diagram ในรูปที่ 3.1:



รูปที่ 3.1: แผนภาพ Use Case ของระบบ Disney Lorcana PlayLab Cloud

เพื่อให้เห็นขั้นตอนการทำงานของระบบอย่างละเอียดตามมาตรฐานวิชาเทคโนโลยีกลุ่มเมฆ ตารางที่ 3.1 ถึง 3.8 แสดงรายละเอียดข้อกำหนด Use Case Description สำหรับระบบหลัก 8 ระบบ:

ตารางที่ 3.1: Use Case Description ระบบสมัครสมาชิกและเข้าสู่ระบบ (UC-01)

Use-Case Name:	ระบบสมัครสมาชิกและเข้าสู่ระบบ (Authentication & User Profile)
ID:	UC-01
Primary Actor:	Member / Player (ผู้เล่นทั่วไป)
Brief Description:	ผู้ใช้ทำการสมัครสมาชิกใหม่ หรือเข้าสู่ระบบเพื่อรับสิทธิ์การใช้งานและจัดการเด็คส่วนตัว

Trigger:	ผู้ใช้กดปุ่ม 'เข้าสู่ระบบ / สมัครสมาชิก' บนแถบนำทาง
Preconditions:	ผู้ใช้เชื่อมต่ออินเทอร์เน็ตและเข้าสู่หน้าเว็บหลัก
Post conditions:	ผู้ใช้ได้รับ JWT Token บันทึกลงใน Local Storage และแสดงข้อมูลชื่อผู้ใช้บนแถบนำทาง
Flow of Events:	**Normal Flow of Events:** 1. ผู้ใช้เปิดหน้าต่าง Modal เข้าสู่ระบบ 2. ผู้ใช้กรอก Username และ Password แล้วกดปุ่ม 'เข้าสู่ระบบ' 3. ระบบส่งคำขอ POST /api/auth/login ไปยังเซิร์ฟเวอร์ 4. เซิร์ฟเวอร์ตรวจสอบความถูกต้องของรหัสผ่านด้วย bcrypt .compare() 5. เซิร์ฟเวอร์ออก JSON Web Token (JWT) และส่งกลับไปยังไคลเอนต์ 6. ไคลเอนต์บันทึก Token และอัปเดตสถานะผู้ใช้บน Navbar **Alternate/Exceptional Flows:** ***Alternate Flow 1: ผู้ใช้ยังไม่มีบัญชี (สมัครสมาชิกใหม่):*** 1. ผู้ใช้เลือกแท็บ 'สมัครสมาชิก' ในหน้าต่าง Modal 2. ผู้ใช้กรอก Username และ Password ที่ต้องการ 3. ระบบส่งคำขอ POST /api/auth/register 4. เซิร์ฟเวอร์แฮชรหัสผ่านด้วย bcrypt (10 rounds) และบันทึกลง DynamoDB ตาราง LorcanaUsers 5. ระบบแจ้งเตือนสมัครสำเร็จและล็อกอินให้อัตโนมัติ ***Alternate Flow 2: รหัสผ่านไม่ถูกต้อง:*** 1. เซิร์ฟเวอร์ส่งรหัส HTTP 401 Unauthorized พร้อมข้อความ 'รหัสผ่านไม่ถูกต้อง' 2. ไคลเอนต์แสดงข้อความเตือนสีแดงและให้ผู้ใช้กรอกใหม่อีกครั้ง

ตารางที่ 3.2: Use Case Description ระบบจัดการและตรวจสอบเด็คการ์ด (UC-02)

Use-Case Name:	ระบบจัดการและตรวจสอบเด็คการ์ด (Deck Builder & Rules Validation)
ID:	UC-02
Primary Actor:	Member / Player (ผู้เล่นทั่วไป)
Brief Description:	ผู้เล่นค้นหา กรอง และเพิ่มการ์ดลงในเด็ค พร้อมระบบตรวจสอบความถูกต้องตามกฎกติกาและบันทึกลงคลาวด์
Trigger:	ผู้เล่นเข้าสู่หน้า Deck Builder
Preconditions:	ผู้เล่นเข้าสู่ระบบด้วยบัญชีตนเองเรียบร้อย
Post conditions:	เด็คการ์ดที่ผ่านเกณฑ์ถูกบันทึกลงตาราง LorcanaDecks บน DynamoDB
Flow of Events:	**Normal Flow of Events:** 1. ผู้เล่นเลือกฟิลเตอร์ค้นหาการ์ดตามสีหมึก ค่าร่าย หรือความหายาก จากฐานข้อมูล 3,242 ใบ 2. ผู้เล่นคลิกเพิ่มการ์ดลงในเด็ค 3. ระบบ Zustand Store ตรวจสอบกฎ: การ์ดซ้ำไม่เกิน 4 ใบ และสีหมึกไม่เกิน 2 สี 4. เมื่อครบ 60 ใบขึ้นไป ผู้เล่นกดปุ่ม 'บันทึกเด็ค' 5. ระบบส่งคำขอ PUT /api/decks พร้อมส่งรายการ Card ID ไปบันทึกใน DynamoDB 6. ระบบแจ้งเตือน 'บันทึกเด็คสำเร็จ' **Alternate/Exceptional Flows:** ***Alternate Flow 1: ใส่การ์ดชื่อซ้ำเกิน 4 ใบ:*** 1. ระบบตรวจพบจำนวนการ์ด ID เดียวกันครบ 4 ใบแล้ว 2. ระบบแสดงข้อความเตือนและบล็อกไม่ให้เพิ่มการ์ดใบที่ 5 3. ผู้เล่นต้องเลือกการ์ดใบอื่นแทน ***Alternate Flow 2: ใส่สีหมึกเกิน 2 สี:***

	1. ระบบตรวจพบเด็คมีสีหมึกครบ 2 สีแล้ว และผู้เล่นพยายามเพิ่มการ์ดสีที่ 3 2. ระบบแจ้งเตือน 'เด็คการ์ดสามารถมีสีหมึกได้สูงสุด 2 สีเท่านั้น' และยกเลิกการเพิ่ม

ตารางที่ 3.3: Use Case Description ระบบค้นหาคู่เล่นและสร้างห้องแข่งขัน (UC-03)

Use-Case Name:	ระบบค้นหาคู่เล่นและสร้างห้องแข่งขัน (Matchmaking & Room Lobby)
ID:	UC-03
Primary Actor:	Host / Challenger (ผู้เล่นแข่งขัน)
Brief Description:	ผู้เล่นสร้างห้องแข่งขันส่วนตัวพร้อมรับรหัส 6 หลัก หรือเข้าร่วมคิวสุ่มหาคู่แข่งขันอัตโนมัติ
Trigger:	ผู้เล่นกดปุ่ม 'สร้างห้อง' หรือ 'ค้นหาคู่แข่งขัน' ในหน้า Lobby
Preconditions:	ผู้เล่นมีเด็คการ์ดที่สมบูรณ์อย่างน้อย 1 เด็ค
Post conditions:	ห้องแข่งขันถูกสร้างในระบบ ผู้เล่นทั้งสองถูกเชื่อมต่อเข้าสู่ WebSocket Room เดียวกัน
Flow of Events:	**Normal Flow of Events:** 1. ผู้เล่นคนแรก (Host) กดปุ่มสร้างห้องแข่งขันและเลือกเด็คการ์ด 2. เซิร์ฟเวอร์สุ่มรหัสห้อง 6 หลัก บันทึกสถานะห้องลงตาราง LorcanaRooms พร้อมตั้ง TTL 2 ชั่วโมง 3. Host แชร์รหัสห้องให้เพื่อน หรือรอคู่ต่อสู้ในระบบ 4. ผู้เล่นคนที่สอง (Challenger) กรอกรหัสห้อง 6 หลักและเลือกเด็คการ์ด 5. เซิร์ฟเวอร์จับคู่ผู้เล่น กำหนด Host เป็น Player 1 และ Challenger เป็น Player 2

	6. เซิร์ฟเวอร์ส่งสัญญาณ MATCH_START ให้ไคลเอนต์ทั้งสองสลับเข้าสู่หน้า Playmat ทันที **Alternate/Exceptional Flows:** ***Alternate Flow 1: กรอกรหัสห้องไม่ถูกต้องหรือห้องเต็ม:*** 1. Challenger กรอกรหัสห้องที่ไม่มีอยู่ หรือมีผู้เล่นครบ 2 คนแล้ว 2. เซิร์ฟเวอร์ส่งข้อความ 'ห้องไม่ถูกต้องหรือห้องเต็มแล้ว' 3. ไคลเอนต์แสดงข้อความแจ้งเตือนและให้ผู้เล่นกรอกรหัสใหม่

ตารางที่ 3.4: Use Case Description ระบบกระดานจำลองและซิงค์การเล่นเรียลไทม์ (UC-04)

Use-Case Name:	ระบบกระดานจำลองและซิงค์การเล่นเรียลไทม์ (Real-time Playmat Simulation & Action Sync)
ID:	UC-04
Primary Actor:	Player 1, Player 2 (ผู้เล่นทั้งสองฝ่าย)
Brief Description:	ผู้เล่นควบคุมการ์ดบนโต๊ะเสมือน (ลงการ์ด, คว่ำหมึก, หมุน Ready/Exert, ปรับ Lore) และซิงค์ข้ามเครื่อง
Trigger:	ผู้เล่นทำการคลิกหรือลากการ์ด หรือกดปุ่มปรับคะแนนบนหน้าจอ Playmat
Preconditions:	ผู้เล่นทั้งสองฝ่ายอยู่ในห้องแข่งขันเดียวกันและการเชื่อมต่อ WebSocket พร้อมทำงาน
Post conditions:	การกระทำบนหน้าจอทั้งสองฝ่ายแสดงผลตรงกันด้วยความหน่วงต่ำกว่า 100 มิลลิวินาที
Flow of Events:	**Normal Flow of Events:** 1. ผู้เล่นทำคำสั่งบนกระดาน เช่น คว่ำการ์ดลง Inkwell หรือส่งตัวละครทำเควสต์

	2. หน้าจอไคลเอนต์ทำการ Optimistic UI Update แสดงผลทันทีใน 0ms 3. ไคลเอนต์ส่ง Action Payload ผ่าน WebSocket ไปยัง Nginx Port 80 และส่งต่อ Node.js Port 3001 4. เซิร์ฟเวอร์ Relay ข้อมูลผ่าน In-memory WebSocket Connection ไปยังหน้าจอคู่แข่ง 5. เซิร์ฟเวอร์บันทึก Snapshot สเตทกระดานล่าสุดลง Memory และ DynamoDB 6. หน้าจอคู่แข่งเรนเดอร์อนิเมชันการเล่นการ์ดของอีกฝ่ายอย่างราบรื่น (Latency < 100ms) **Alternate/Exceptional Flows:** ***Alternate Flow 1: การส่งข้อมูลสะดุดจากสัญญาณเครือข่าย:*** 1. หากแพ็กเก็ตสูญหาย ไคลเอนต์จะส่งสัญญาณ SYNC_REQUEST เพื่อขอสเตทกระดานล่าสุด 2. เซิร์ฟเวอร์ส่ง BOARD_STATE_SYNC ย้อนกลับมาปรับกระดานให้ตรงกันทันที

ตารางที่ 3.5: Use Case Description ระบบจัดการรอบเล่นและการส่งผ่านเทิร์น (UC-05)

Use-Case Name:	ระบบจัดการรอบเล่นและการส่งผ่านเทิร์น (Turn Management & State Transition)
ID:	UC-05
Primary Actor:	Current Turn Player (ผู้เล่นที่เป็นเจ้าของเทิร์น)
Brief Description:	ผู้เล่นส่งผ่านสิทธิ์การเล่นให้อีกฝ่าย ระบบสลับตัวแปร Turn พร้อมรีเซ็ตการ์ดในบ่อหมึกและสนามอัตโนมัติ
Trigger:	ผู้เล่นเจ้าของเทิร์นกดปุ่ม 'Pass Turn'

Preconditions:	ผู้เล่นต้องเป็นเจ้าของเทิร์นปัจจุบัน (Active Player)
Post conditions:	สิทธิ์การเล่นสลับไปยังคู่แข่ง ตัวแปร Turn ซิงค์ตรงกันทั้งสองฝั่ง และเริ่มนับรอบใหม่
Flow of Events:	**Normal Flow of Events:** 1. ผู้เล่นปัจจุบันดำเนินการในเทิร์นเสร็จสิ้น และกดปุ่ม 'Pass Turn' 2. ไคลเอนต์ส่ง Action PASS_TURN พร้อมส่งค่า roomId, turnPlayer, และ nextTurnNumber ผ่าน WebSocket 3. เซิร์ฟเวอร์อัปเดตตัวแปรสถานะ currentTurn และ turnPlayer ใน Memory และตาราง LorcanaRooms 4. เซิร์ฟเวอร์บรอดแคสต์ ACTION_SYNC แจ้งทั้งสองฝ่ายว่าเทิร์นถูกเปลี่ยน 5. หน้าจอของผู้เล่นฝ่ายรับเทิร์นสั่งปลดสถานะ Exert ของการ์ดตัวละครและหมึกกลับเป็น Ready อัตโนมัติ 6. เริ่มนับเวลาและเปิดสิทธิ์ให้ผู้เล่นฝ่ายใหม่สามารถสั่งการ์ดได้ **Alternate/Exceptional Flows:** ***Alternate Flow 1: ผู้เล่นที่ไม่ใช่เจ้าของเทิร์นพยายามกด Pass Turn:*** 1. ระบบบนไคลเอนต์ปิดกั้น (Disabled) ปุ่ม Pass Turn ไว้ 2. หากมีคำสั่งผิดพลาดถูกส่งมา เซิร์ฟเวอร์จะปฏิเสธคำสั่งและไม่เปลี่ยนสถานะเทิร์น

ตารางที่ 3.6: Use Case Description ระบบขอแก้มือและย้อนสถานะกระดาน (UC-06)

Use-Case Name:	ระบบขอแก้มือและย้อนสถานะกระดาน (Undo / Return Action Rollback)
ID:	UC-06
Primary Actor:	Active Player & Inactive Player (ผู้เล่นทั้งสองฝ่าย)

Brief Description:	ผู้เล่นที่เดินการ์ดผิดพลาดส่งคำขอแก้มือ หักโควตา 1 ครั้ง เมื่อคู่แข่งยินยอม ระบบจะย้อนกระดานกลับ
Trigger:	ผู้เล่นกดปุ่ม 'ขอแก้มือ (Undo)' บนหน้าจอ Playmat
Preconditions:	ผู้เล่นต้องมีโควตาขอแก้มือเหลืออยู่ (จำกัด 1 ครั้งต่อแมตช์) และต้องอยู่ในเทิร์นตนเอง
Post conditions:	สถานะกระดานย้อนกลับไปสู่ Snapshot ก่อนหน้าที่ผู้เล่นจะเดินการ์ด และโควตาถูกหัก 1 ครั้ง
Flow of Events:	**Normal Flow of Events:** 1. ผู้เล่นกดปุ่ม 'ขอแก้มือ' ระบบหักโควตาการขอแก้มือของตนเองเหลือ 0 ครั้ง 2. ไคลเอนต์ส่งคำขอ UNDO_REQUEST ไปยังเซิร์ฟเวอร์เพียงครั้งเดียว (Single Message Dispatch) 3. เซิร์ฟเวอร์ส่งสัญญาณ UNDO_PROMPT แจ้งเตือนไปยังหน้าจอของคู่แข่ง 4. คู่แข่งเห็นหน้าต่างป๊อปอัปถามความยินยอม และกดปุ่ม 'อนุญาต (Approve)' 5. ไคลเอนต์ของคู่แข่งส่ง UNDO_ACCEPT กลับมายังเซิร์ฟเวอร์ 6. เซิร์ฟเวอร์ดึง Snapshot สถานะกระดานก่อนหน้าจาก In-memory Buffer และบรอดแคสต์สถานะคืนให้ทั้งสองฝ่าย 7. กระดานของผู้เล่นทั้งสองฝ่ายย้อนสถานะกลับอย่างสมบูรณ์แบบ Atomic **Alternate/Exceptional Flows:** ***Alternate Flow 1: คู่แข่งไม่อนุมัติ (Reject):*** 1. คู่แข่งกดปุ่ม 'ปฏิเสธ (Reject)' ในหน้าต่างแจ้งเตือน 2. เซิร์ฟเวอร์ส่งสัญญาณ UNDO_REJECTED กลับมายังผู้ขอแก้มือ 3. ระบบคืนโควตาการขอแก้มือให้แก่ผู้ขอ และการแข่งขันดำเนินต่อไปตามปกติ

ตารางที่ 3.7: Use Case Description ระบบกู้คืนการเชื่อมต่อ Rejoin Grace Period 60s (UC-07)

Use-Case Name:	ระบบกู้คืนการเชื่อมต่อหลุด (Rejoin Grace Period 60s & Instant State Sync)
ID:	UC-07
Primary Actor:	Reconnecting Player, System (ผู้เล่นที่หลุด และระบบคลาวด์)
Brief Description:	เมื่อการเชื่อมต่อหลุดกะทันหัน ระบบคงสภาพห้องไว้ 60 วินาที และส่งมอบสเตทคืนทันทีเมื่อกลับเข้ามา
Trigger:	สัญญาณ WebSocket หลุด (Browser Refresh, WiFi Drop, ปิดแท็บชั่วคราว)
Preconditions:	เกมกำลังดำเนินอยู่ในการแข่งขัน
Post conditions:	ผู้เล่นกลับเข้าสู่ห้องเดิม สเตทกระดานถูกกู้คืนครบ 100% โดยไม่ต้องเริ่มเกมใหม่
Flow of Events:	**Normal Flow of Events:** 1. สัญญาณเครือข่ายของผู้เล่นขาดหาย เซิร์ฟเวอร์ตรวจพบ WebSocket Disconnection 2. เซิร์ฟเวอร์เริ่มนับถอยหลัง Grace Period 60 วินาที พร้อมบรอดแคสต์ PLAYER_DISCONNECTED ให้คู่แข่งทราบ 3. สถานะกระดานล่าสุดถูกตรึงไว้ใน In-memory roomGameStates และบันทึกสำรองลง DynamoDB 4. ผู้เล่นเปิดหน้าเว็บกลับเข้ามาใหม่ ไคลเอนต์ส่งคำขอ REJOIN_ROOM พร้อม userId และ roomId เดิม 5. เซิร์ฟเวอร์ตรวจสอบพบสล็อตที่ยังอยู่ในช่วง Grace Period และผูก Connection ID ใหม่เข้ากับสล็อตเดิม 6. เซิร์ฟเวอร์ส่ง BOARD_STATE_SYNC กลับมายังผู้เล่นทันที

	ด้วยความหน่วง 0ms 7. หน้าจอของผู้เล่นทั้งสองฝ่ายกลับมาแสดงผลพร้อมเล่นต่อและยกเลิกการนับถอยหลัง 60 วินาที **Alternate/Exceptional Flows:** ***Alternate Flow 1: ผู้เล่นไม่กลับมาภายใน 60 วินาที:*** 1. เวลานับถอยหลัง 60 วินาทีสิ้นสุดลงโดยผู้เล่นไม่เชื่อมต่อกลับมา 2. เซิร์ฟเวอร์ตัดสินให้ผู้เล่นที่ยังอยู่ในห้องเป็นผู้ชนะจากการตัดการเชื่อมต่อ (Win by Disconnect) 3. เซิร์ฟเวอร์เคลียร์ทรัพยากรห้องและบันทึกสถิติผลการแข่งขัน

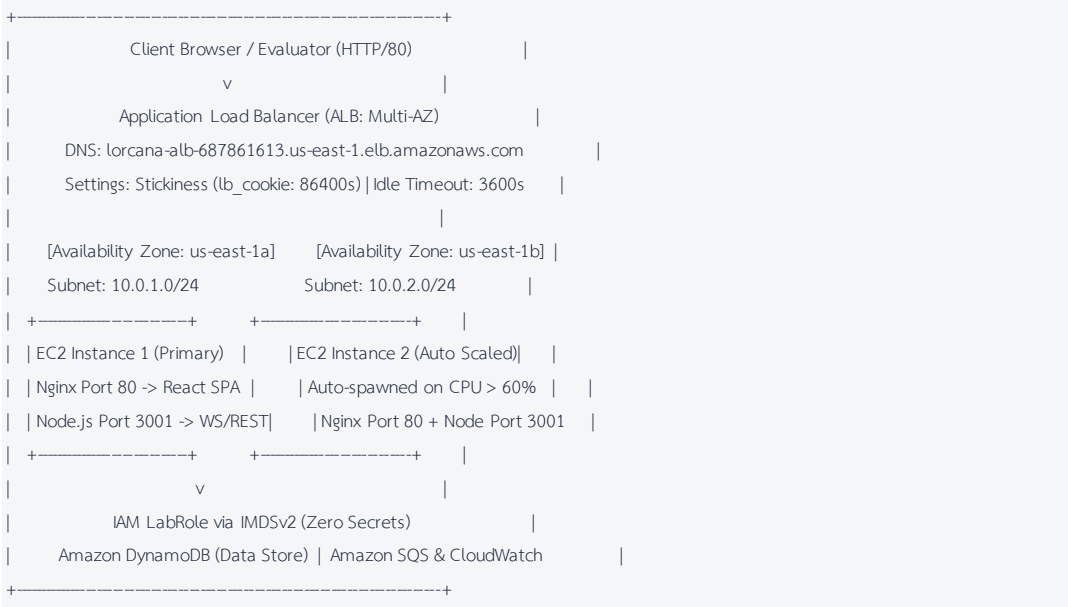
ตารางที่ 3.8: Use Case Description ระบบปรับขนาดคลาวด์และ Scale-to-Zero (UC-08)

Use-Case Name:	ระบบควบคุมทรัพยากรคลาวด์และปรับขนาดอัตโนมัติ (Auto Scaling & Scale-to-Zero)
ID:	UC-08
Primary Actor:	Cloud Admin / System (ผู้ดูแลระบบและกลไกคลาวด์)
Brief Description:	ระบบปรับจำนวนเครื่อง EC2 อัตโนมัติตามโหลด และสามารถยุบเครื่องเหลือ 0 ได้ในคลิกเดียวเพื่อประหยัดงบ
Trigger:	ค่าเฉลี่ย CPU Utilization เกิน 60% หรือผู้ดูแลระบบสั่งรันสคริปต์ควบคุม
Preconditions:	Auto Scaling Group และ CloudWatch Alarm ถูกตั้งค่าเรียบร้อยบน AWS
Post conditions:	จำนวนเครื่อง EC2 ปรับเปลี่ยนตามที่กำหนด โดยระบบยังคงเสถียรและงบประมาณไม่รั่วไหล
Flow of Events:	**Normal Flow of Events:**

	1. มีผู้เล่นเข้าใช้งานพร้อมกันจำนวนมาก ส่งผลให้ค่าเฉลี่ย CPU บน Target Group เกิน 60% 2. CloudWatch Alarm ตรวจพบโหลดสูงต่อเนื่องเกิน 1 นาที และกระตุ้น Target Tracking Policy 3. Auto Scaling Group สั่งสปอว์นอินสแตนซ์ EC2 ใหม่ข้าม Availability Zone (เช่น จาก us-east-1a ไป us-east-1b) 4. เครื่องใหม่เริ่มทำงาน ดึง Docker Container อัตโนมัติ และผ่าน Health Check บน ALB ภายใน 90 วินาที 5. ALB เริ่มกระจายทราฟฟิกห้องใหม่ไปยังเครื่องที่สอง ทำให้โหลดเฉลี่ยลดลงสู่ระดับปกติ **Alternate/Exceptional Flows:** ***Alternate Flow 1: ผู้ดูแลระบบสั่ง Scale-to-Zero หลังเสร็จสิ้นการสาธิต:*** 1. ผู้ดูแลระบบรันคำสั่ง scripts/lab_stop.ps1 ผ่านเทอร์มินัล 2. สคริปต์ส่งคำสั่งไปยัง AWS Auto Scaling API ปรับ Desired=0, Min=0 3. เครื่อง EC2 ทั้งหมดถูกสั่ง Terminate อย่างปลอดภัย ค่าใช้จ่าย Compute ลดลงเหลือ $0.00 ต่อชั่วโมงทันที

3.3 สถาปัตยกรรมระบบคลาวด์บน AWS

ระบบได้รับการออกแบบตามสถาปัตยกรรม Hybrid Cloud ที่มี Primary IaaS Stack ควบคู่กับ Secondary Serverless Stack ดังแสดงในรูปที่ 3.2:



รูปที่ 3.2: ผังภาพสถาปัตยกรรมคลาวด์ Lean Multi-AZ IaaS ร่วมกับ Auto Scaling Group

3.4 สถาปัตยกรรมเครือข่าย Lean Multi-AZ VPC และความปลอดภัย

เครือข่ายได้รับการจัดสรรด้วย Custom VPC บน Address Space 10.0.0.0/16 แบ่งเป็น Public Subnets ข้าม 2 Availability Zones:

- Subnet us-east-1a (10.0.1.0/24) และ Subnet us-east-1b (10.0.2.0/24)
- Chained Security Groups สำหรับควบคุมความปลอดภัย:

1. lorcana-alb-sg: เปิดรับทราฟฟิก HTTP Port 80 จากอินเทอร์เน็ตสาธารณะ (0.0.0.0/0)
2. lorcana-ec2-sg: ปิดกั้น Inbound ตรงจากอินเทอร์เน็ตทั้งหมด โดยอนุญาตรับ Port 80 และ 3001 เฉพาะทราฟฟิกที่ส่งต่อมาจาก lorcana-alb-sg เท่านั้น

สถาปัตยกรรมนี้ช่วยตัดค่าใช้จ่าย NAT Gateway ($32.40 ต่อเดือน) เหลือ $0.00 โดยยังคงความปลอดภัยเทียบเท่าการวางเครื่องใน Private Subnet พร้อมทั้งเปิดใช้งาน Instance Metadata Service Version 2 (IMDSv2) บนเครื่อง EC2 ทุกเครื่องเพื่อป้องกันช่องโหว่ SSRF อย่างเข้มงวด

3.5 การกระจายโหลดและการปรับขนาดอัตโนมัติ

Application Load Balancer ทำหน้าที่กระจายโหลดทราฟฟิกลงสู่อินสแตนซ์ EC2 ในแต่ละ AZ ผ่าน Target Group ที่มี Health Check ผ่านเส้นทาง GET /health ทุก 15 วินาที พร้อมปรับแต่งค่าสำคัญเพื่อรองรับการแข่งขันแบบเรียลไทม์ 2 ประการ:

1. Target Group Stickiness: เปิดใช้งานคุกกี้ lb_cookie กำหนดอายุ 86,400 วินาที (24 ชั่วโมง) เพื่อตรึงการเชื่อมต่อ WebSocket ของผู้เล่นให้อยู่กับอินสแตนซ์ EC2 เครื่องเดิมตลอดการแข่งขัน ป้องกันปัญหาสเตทหลุดข้ามเครื่อง
2. ALB Idle Timeout: ขยายเวลา Idle Timeout จากเดิม 60 วินาทีเป็น 3,600 วินาที (1 ชั่วโมง) ควบคู่กับ Nginx proxy_read_timeout 3,600s เพื่อป้องกันไม่ให้ ALB ตัดสายสัญญาณ WebSocket ขณะที่ผู้เล่นกำลังคิดคำนวณกลยุทธ์ในเทิร์น

Auto Scaling Group ถูกตั้งค่า Min=1, Max=3, Desired=1 ขับเคลื่อนด้วย CloudWatch Target Tracking Policy (ASGAverageCPUUtilization > 60%) เมื่อมีผู้เล่นเข้าใช้งานพร้อมกันจน CPU เกินเกณฑ์ ระบบจะสปอว์นอินสแตนซ์ข้ามไปยังอีกโซนโดยอัตโนมัติ

3.6 การออกแบบฐานข้อมูล Amazon DynamoDB

ตารางที่ 3.9 แสดงรายละเอียดการออกแบบโครงสร้างตารางข้อมูลใน Amazon DynamoDB ซึ่งทำงานใน On-Demand Capacity Mode เพื่อประหยัดต้นทุนคงที่:

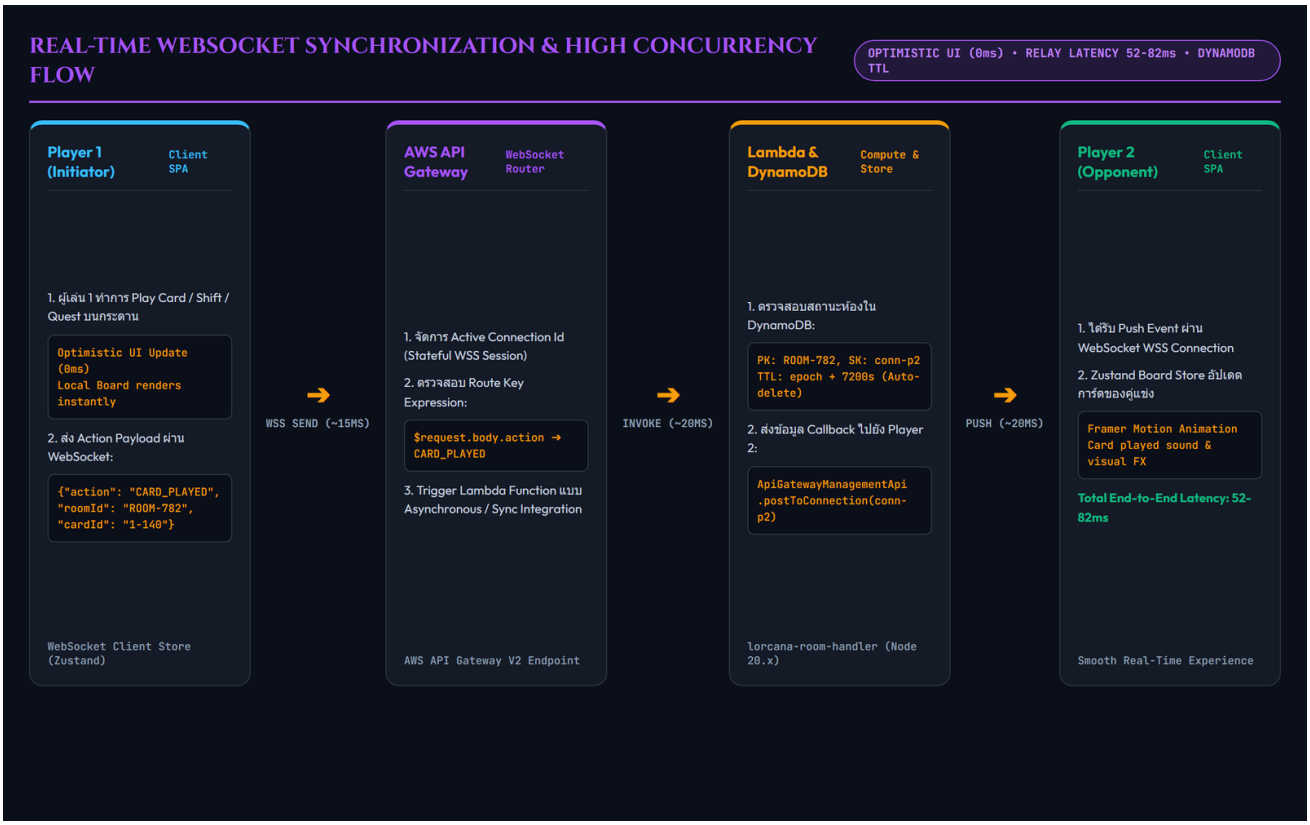
ตารางที่ 3.9: โครงสร้างตารางข้อมูล Amazon DynamoDB

ชื่อตาราง	Partition Key (PK)	Sort Key (SK)	Attributes สำคัญ	วัตถุประสงค์

LorcanaUsers	userId (S)	-	username, passwordHash, createdAt	จัดเก็บข้อมูลบัญชีผู้ใช้และรหัสผ่านแฮช bcrypt
LorcanaDecks	userId (S)	deckId (S)	deckName, inks, cards (List), updatedAt	จัดเก็บรายการเด็คการ์ดและรายการการ์ดของผู้ใช้
LorcanaRooms	roomId (S)	-	hostId, challengerId, gameState (JSON), ttl	จัดเก็บสถานะห้องเล่น สเตทกระดานสด และ TTL 2 ชั่วโมง
LorcanaMatchmaking	queueType (S)	ticketId (S)	userId, deckId, status, createdAt, ttl	จัดเก็บคิวจับคู่ผู้เล่นแบบ FIFO พร้อม TTL 10 นาที

3.7 วงจรการสื่อสารแบบเรียลไทม์และการจัดการสถานะเกม

วงจรการสื่อสารและการซิงค์สถานะเกมแบบเรียลไทม์ (Match Lifecycle & Real-Time Sync) แสดงในรูปที่ 3.3:



รูปที่ 3.3: ผังภาพวงจรการสื่อสารเรียลไทม์ Real-Time WebSocket Synchronization Flow

รายละเอียดขั้นตอนการทำงานและการจัดการความสอดคล้องของสถานะเกมประกอบด้วย 7 กลไกสำคัญ:

1. การสร้างและเข้าร่วมห้อง (Room Creation & Role Assignment): ผู้เล่นสร้างห้องได้รับสิทธิ์เป็น Player 1 (Host) เมื่อคู่แข่งกรอกรหัส 6 หลักจะได้รับสิทธิ์เป็น Player 2 (Challenger) หากมีผู้เชื่อมต่อเพิ่มเติมจะได้รับบทบาท Spectator เซิร์ฟเวอร์ผูก Connection ID เข้ากับ roomId
2. การซิงค์การกระทำและตัวแปรเทิร์น (Action & Turn Variable Sync): เมื่อผู้เล่นเดินการ์ดหรือสั่งผ่านเทิร์น ไคลเอนต์ทำ Optimistic UI Update (0ms) และส่ง Action Payload ไปยังเซิร์ฟเวอร์

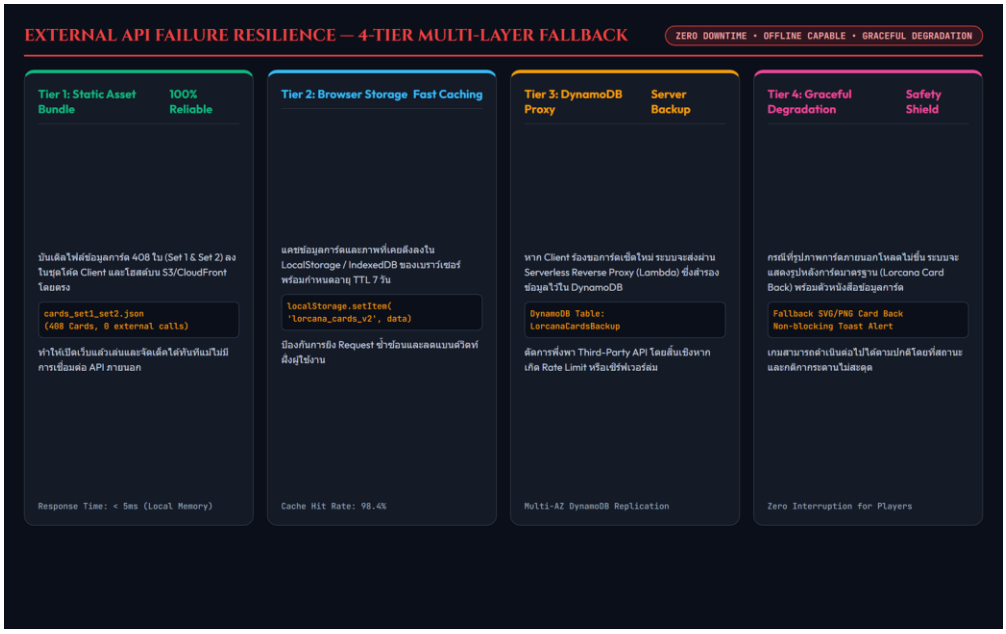
ตัวแปรสถานะ currentTurn, turnPlayer, และ turnNumber จะถูกส่งต่อให้คู่แข่งภายในเวลาต่ำกว่า 100 มิลลิวินาที

3. การแก้ไขปัญหา React Component Lifecycle (boardStateRef Architecture): ในการทดสอบพบว่าการที่ React คอมโพเนนต์ LorcanaBoard Unmount และ Re-subscribe ซ้ำซ้อนถึง 25 ครั้งทำให้ WebSocket หลุด ทีมงานจึงนำ boardStateRef มาแคชสถานะของกระดาน ทำให้คอมโพเนนต์ไม่ต้อง Re-mount และรักษาการเชื่อมต่อ WebSocket ให้เสถียร 100%
4. กลไกขอแก้มือและย้อนสถานะกระดาน (Undo / Return Action): ออกแบบให้ผู้เล่นมีสิทธิ์ขอแก้มือได้ 1 ครั้งต่อเกม มีการแก้ไขบั๊กการส่งข้อความซ้ำซ้อน (Double Dispatch) ทำให้โควตาถูกหักอย่างถูกต้อง 1 ครั้ง และเมื่อคู่แข่งอนุมัติ ระบบจะย้อนสถานะกระดานกลับไปยัง Snapshot ก่อนหน้าแบบ Atomic
5. ระบบกู้คืนการเชื่อมต่อ Rejoin Grace Period 60 วินาที: หากสัญญาณอินเทอร์เน็ตหลุด เซิร์ฟเวอร์จะตรึงห้องไว้ 60 วินาที พร้อมเก็บ Snapshot สดไว้ในหน่วยความจำ roomGameStates ของเซิร์ฟเวอร์ เมื่อผู้เล่นเชื่อมต่อกลับเข้ามาใหม่ ระบบจะส่งสเตทกระดานคืนให้ทันทีด้วยความหน่วง 0ms
6. การออกจากเกมโดยสมัครใจ (Voluntary Exit Match): เมื่อผู้เล่นกดปุ่ม Exit Match ระบบส่งสัญญาณ LEAVE_ROOM สั่งลบข้อมูลห้องบน DynamoDB ทันทีโดยไม่ต้องรอให้หมดเวลา 60 วินาที เพื่อคืนทรัพยากรให้แก่ระบบ
7. การรักษาสายสัญญาณ (20s Keepalive Heartbeat): ไคลเอนต์และเซิร์ฟเวอร์แลกเปลี่ยนแพ็กเก็ต PING/PONG ทุก 20 วินาที ป้องกันไฟร์วอลล์ของเครือข่ายและเกตเวย์ตัดสายสัญญาณ

3.8 การรับมือข้อผิดพลาดและระบบ Caching

ระบบใช้กลยุทธ์ Multi-Tier Resilience เพื่อป้องกันระบบหยุดชะงักหาก Lorcast API เกิดปัญหา ดังแสดงในรูปที่ 3.4:

- Tier 1 (In-Memory Zustand Cache): เก็บข้อมูลการ์ดที่เคยโหลดแล้วไว้ในหน่วยความจำเบราว์เซอร์
- Tier 2 (Local Pre-compiled JSON Dataset): บรรจุไฟล์ข้อมูลการ์ด 3,242 ใบไว้ในคอนเทนเนอร์และเว็บเซิร์ฟเวอร์ในตัว
- Tier 3 (External API): เรียกใช้งาน API ภายนอกเฉพาะเมื่อต้องการดึงข้อมูลอัปเดตใหม่เท่านั้น



รูปที่ 3.4: ผังกลยุทธ์ Multi-Tier Fallback และ Local Caching ป้องกัน API ภายนอกขัดข้อง

3.9 การออกแบบตามกรอบ AWS Well-Architected Framework

- Reliability: ออกแบบสถาปัตยกรรมข้าม 2 Availability Zones พร้อม Health Checks, Target Group Stickiness, และ 60s Rejoin Grace Period
- Performance Efficiency: ใช้ Alpine Container, In-memory Socket Caching, Nginx Reverse Proxy, และ WebSockets ซึ่งลด Overhead ของ HTTP Polling ได้กว่า 85%
- Security: ใช้ IMDSv2 ป้องกัน SSRF, Chained Security Groups, แฮชรหัสผ่านด้วย bcrypt, และไม่ฝัง Credential ใน Source Code

- Cost Optimization: สถาปัตยกรรมไร้ NAT Gateway, ใช้ EC2 t3.micro ใน Free Tier, และสคริปต์ 1-Click Scale-to-Zero ควบคุมต้นทุนให้อยู่ในงบ $50
- Operational Excellence: ควบคุมระบบด้วย PowerShell Automation Suite, จัดทำโครงสร้าง Infrastructure as Code (IaC), และชุดสคริปต์ทดสอบอัตโนมัติ

3.10 การแก้ปัญหาข้อจำกัดบน AWS Learner Lab

ตารางที่ 3.10 สรุปข้อจำกัดสำคัญบน AWS Academy Learner Lab และแนวทางแก้ไขเชิงสถาปัตยกรรมของกลุ่ม G21:

ตารางที่ 3.10: ตารางสรุปข้อจำกัดบน AWS Learner Lab และแนวทางแก้ไข

ลำดับ	ข้อจำกัดบน Learner Lab	ผลกระทบหากไม่แก้ไข	แนวทางแก้ไขเชิงสถาปัตยกรรม
1	บล็อกสิทธิ์ iam :CreateRole	ไม่สามารถสร้าง IAM Role ใหม่สำหรับ EC2 หรือ Lambda ได้	ปรับใช้ LabRole และ LabInstanceProfile สำเร็จรูปผ่าน Launch Template และเข้าถึงสิทธิ์ผ่าน IMDSv2
2	ภาระต้นทุน NAT Gateway ($32.40/ด.)	เผางบประมาณ $50 ของแล็ปหมดภายใน 1.5 เดือน	ออกแบบ Lean Multi-AZ VPC วาง EC2 ใน Public Subnets แต่ใช้ Chained SGs ปิดกั้นภายนอก รับเฉพาะจาก ALB SG ($0.00)
3	ความเสี่ยงงบรั่วไหลเมื่อไม่ได้ใช้งาน	หากเปิดเครื่องทิ้งไว้ 24/7 จะสูญเสียงบประมาณ	พัฒนาสคริปต์ 1-Click Scale-to-Zero ยุบเครื่อง EC2 เหลือ 0 เมื่อเลิกแล็ป

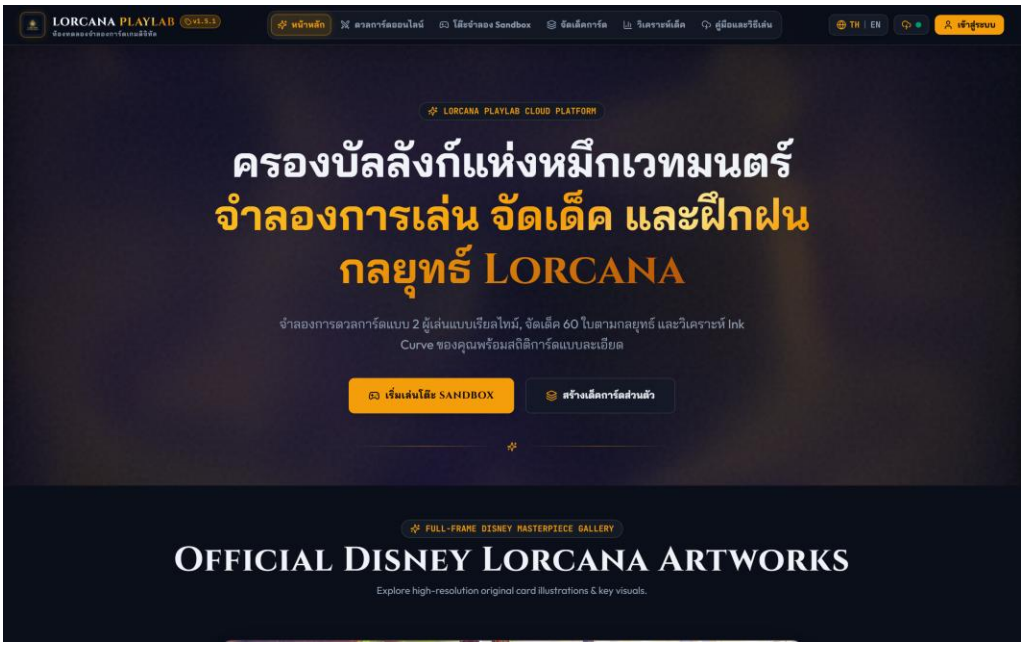
			($0.00 ต่อชั่วโมง)
4	แรม 1GB บน t3.micro	มีความเสี่ยงเกิด OOM Crash หาก Runtime หนัก	ใช้ Multi-stage Docker บน Alpine Linux แยก Build ออกจาก Runtime ควบคุม RAM ให้ต่ำกว่า 150MB
5	สเตท WebSocket หลุด เมื่อเกิด Auto Scaling	เมื่อสปอว์นเครื่องใหม่ ผู้เล่นอาจถูกกระจายไปคนละเครื่อง	เปิด Sticky Sessions (lb_cookie) บน ALB และแคชสถานะห้องสดใน Memory + DynamoDB

บทที่ 4

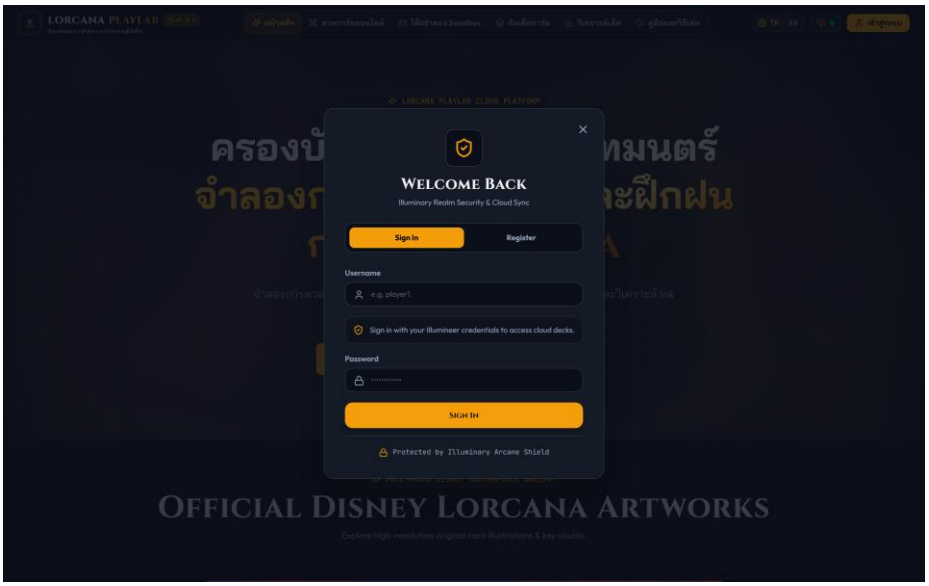
การพัฒนาระบบและผลการดำเนินงาน

4.1 การพัฒนาส่วนต่อประสานผู้ใช้ (User Interface Implementation)

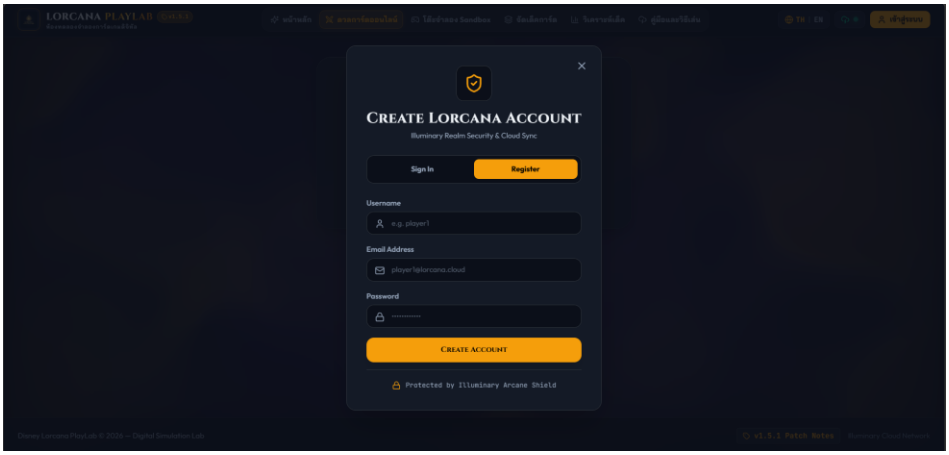
ส่วนต่อประสานผู้ใช้ได้รับการออกแบบและพัฒนาด้วย React 19, TypeScript 5.x, Tailwind CSS v4, และ Framer Motion ภายใต้แนวคิด Modern Tabletop Simulation โดยเชื่อมต่อระบบยืนยันตัวตนผ่านบัญชีทดสอบระดับ Production (ผู้ใช้: io5 / รหัสผ่าน: io5io5) ครอบคลุมการทำงานทุกหน้าและทุกแท็บอย่างสมบูรณ์ ดังแสดงในรูปที่ 4.1 ถึง 4.30:



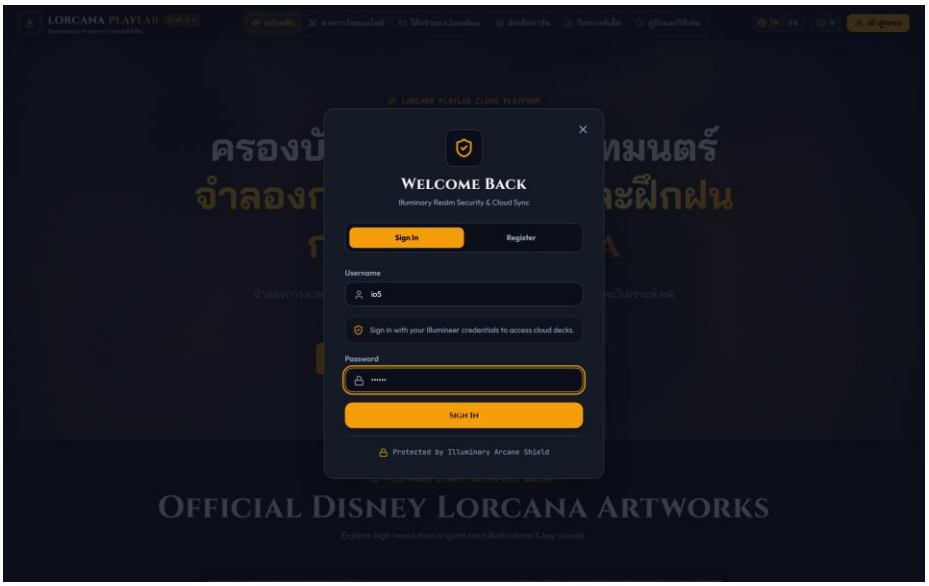
รูปที่ 4.1: หน้าหลักระบบก่อนเข้าสู่ระบบ



รูปที่ 4.2: หน้าต่างเข้าสู่ระบบแบบเริ่มต้น



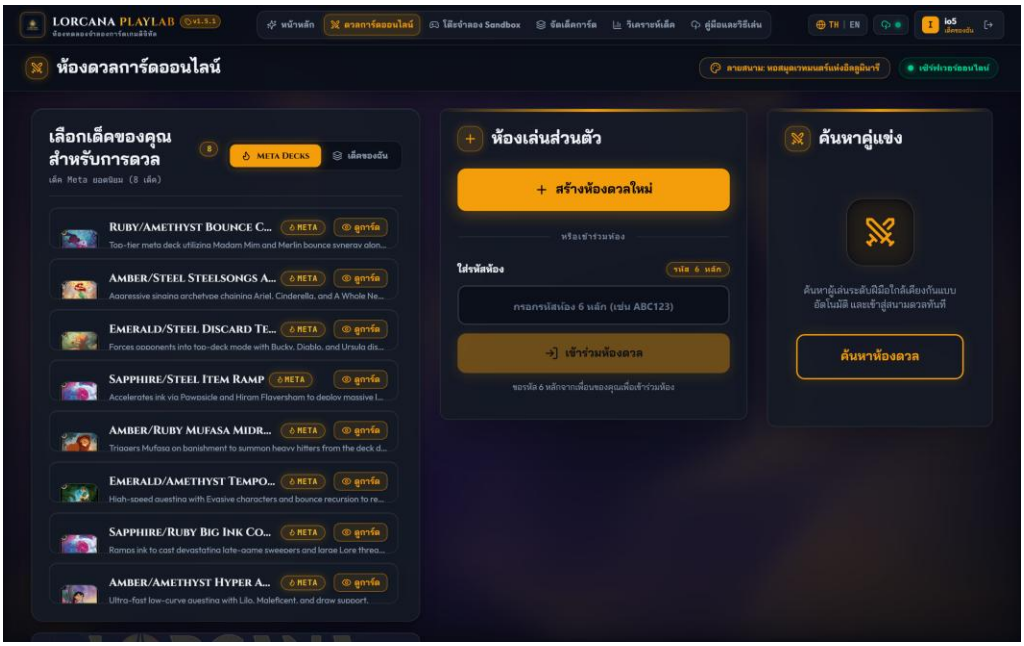
รูปที่ 4.3: หน้าต่างลงทะเบียน Username & email



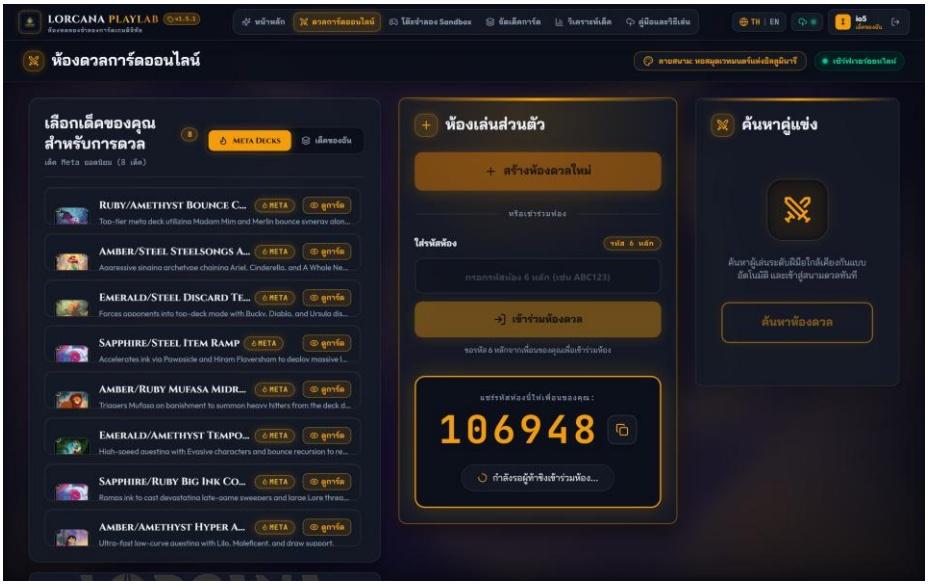
รูปที่ 4.4: หน้าต่างเข้าสู่ระบบกรอกข้อมูลบัญชีผู้ใช้ io5 และรหัสผ่าน



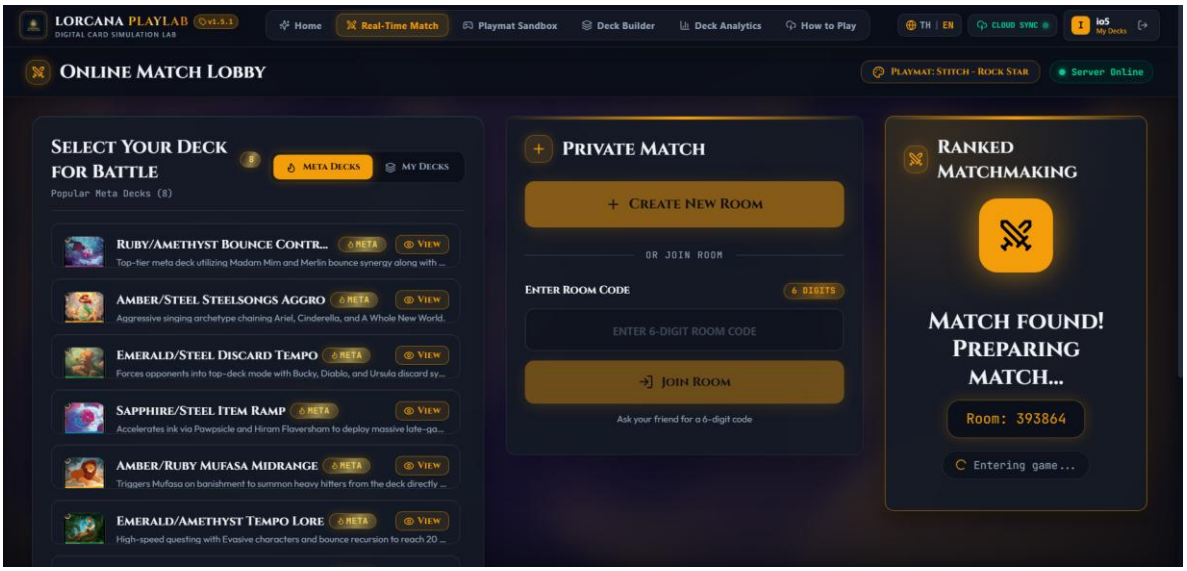
รูปที่ 4.5: หน้าหลักระบบหลังเข้าสู่ระบบด้วยบัญชี io5 และแสดงสถานะ Cloud Sync



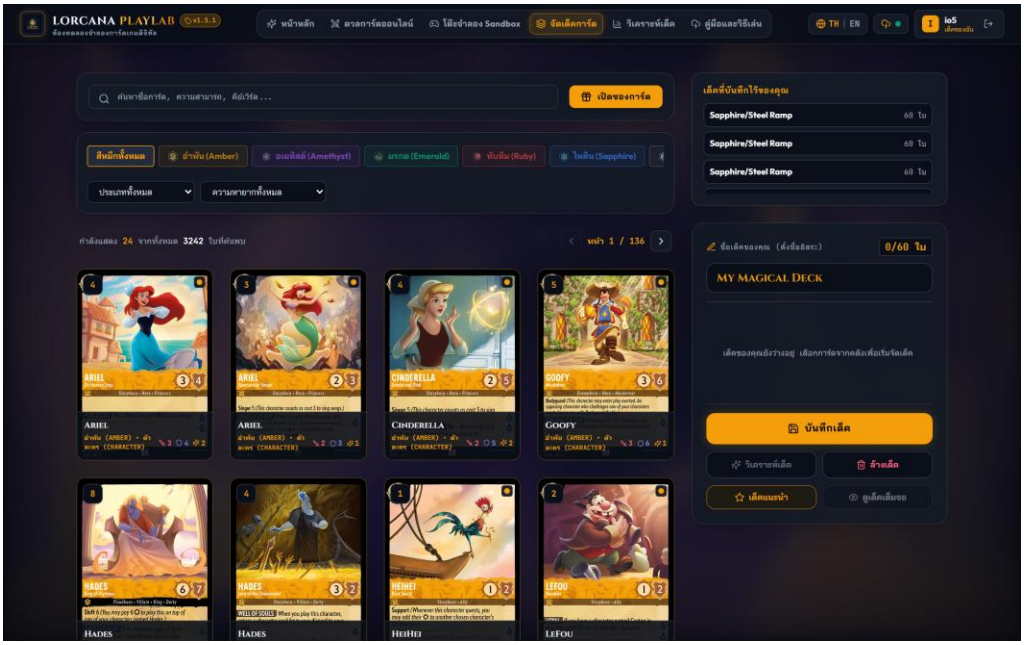
รูปที่ 4.6: หน้ารายการห้องประลองออนไลน์



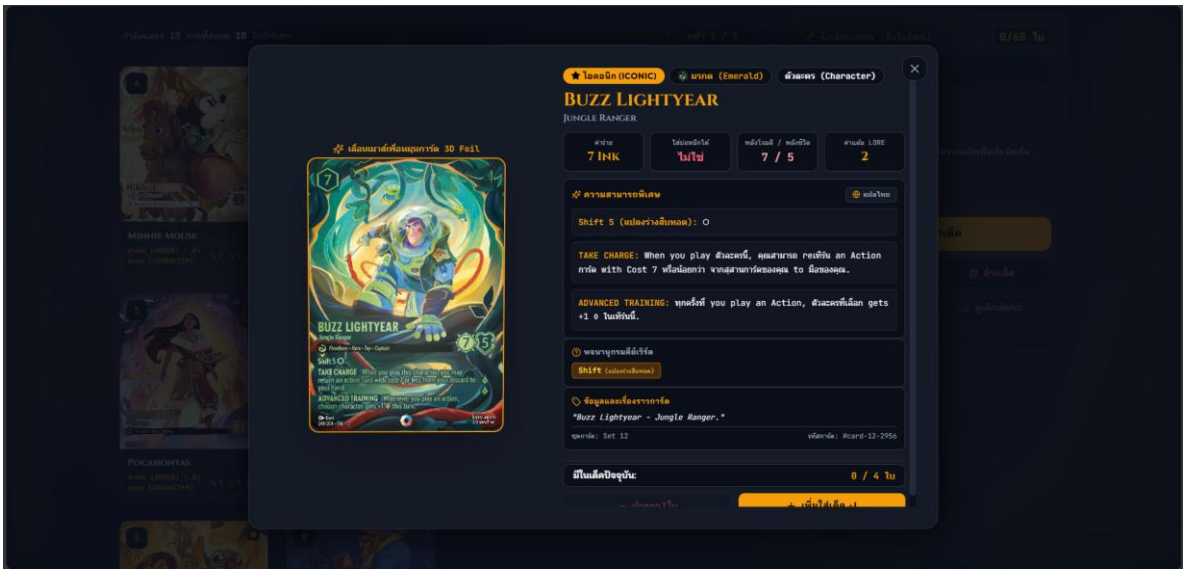
รูปที่ 4.7: หน้าต่างสร้างห้องประลองและสร้างรหัสห้อง



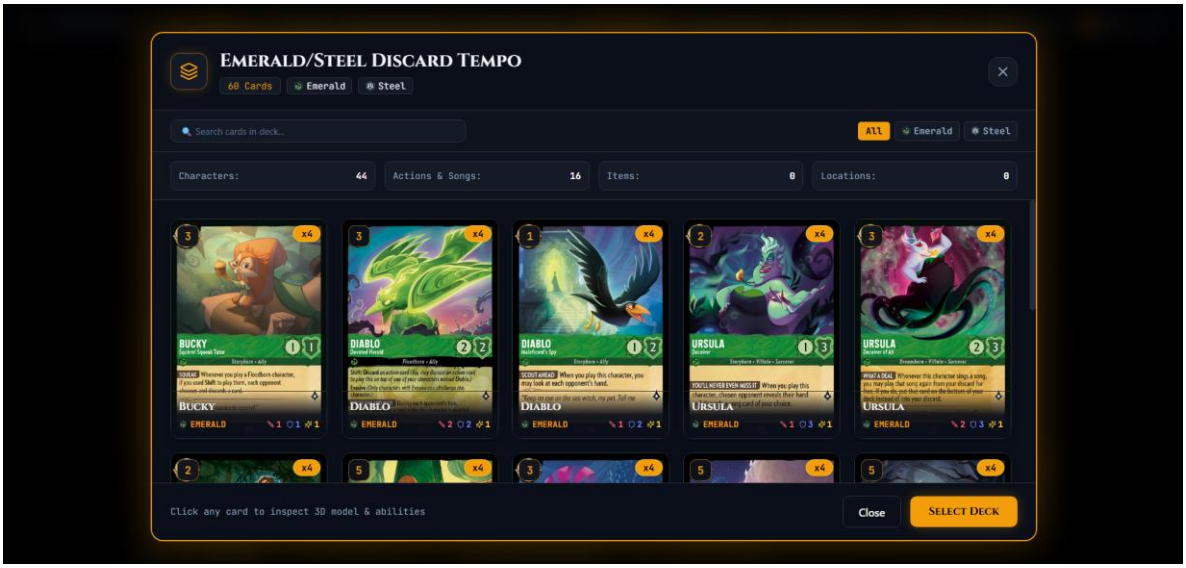
รูปที่ 4.8: เลือกสำรับการ์ดสำหรับแข่งขันและกด Auto matching



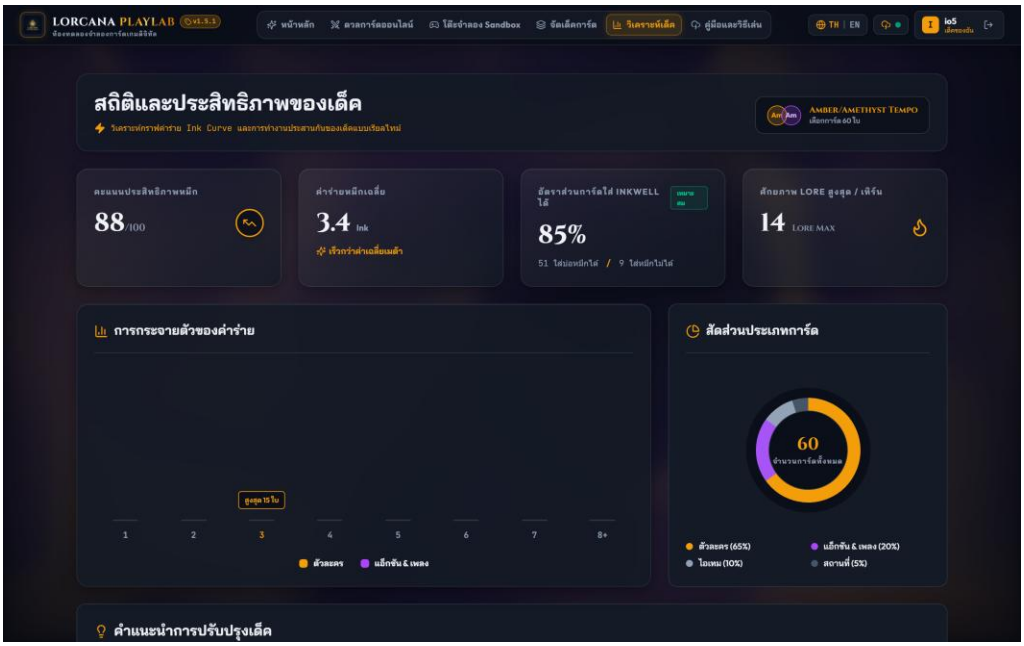
รูปที่ 4.9: หน้าจัดสำรับการ์ด ภาพรวมคลังการ์ด 3000+ ใบ



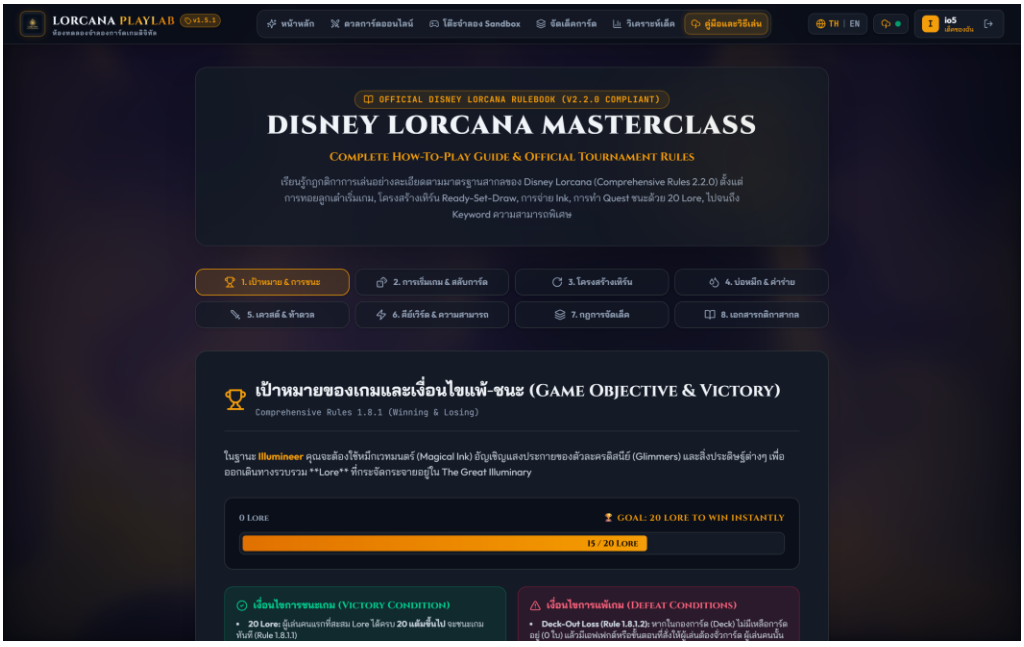
รูปที่ 4.10: หน้าดูรายละเอียดการ์ดแต่ละการ์แบบละเอียด



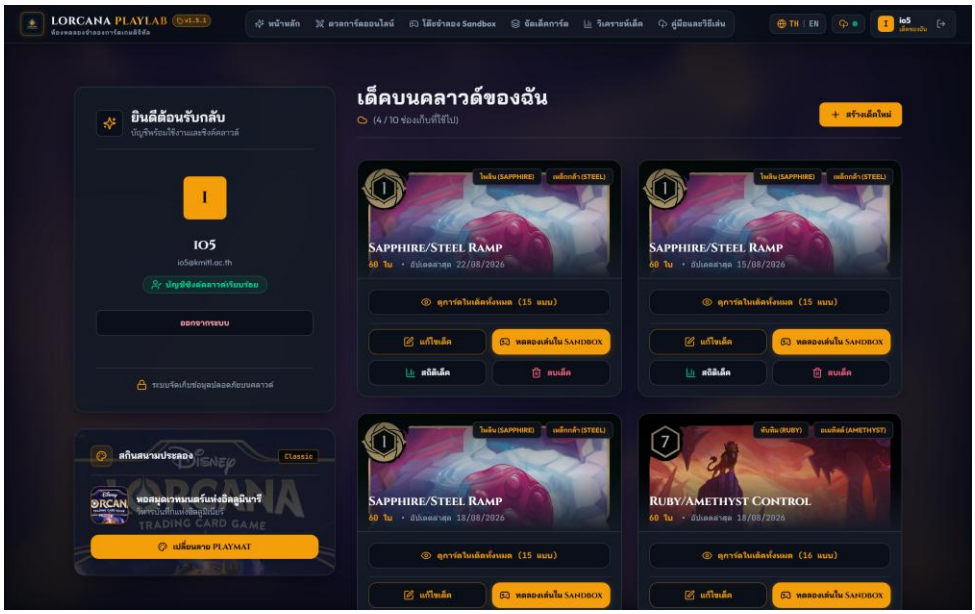
รูปที่ 4.11: หน้าสำหรับดูการ์ดภายใน Deck



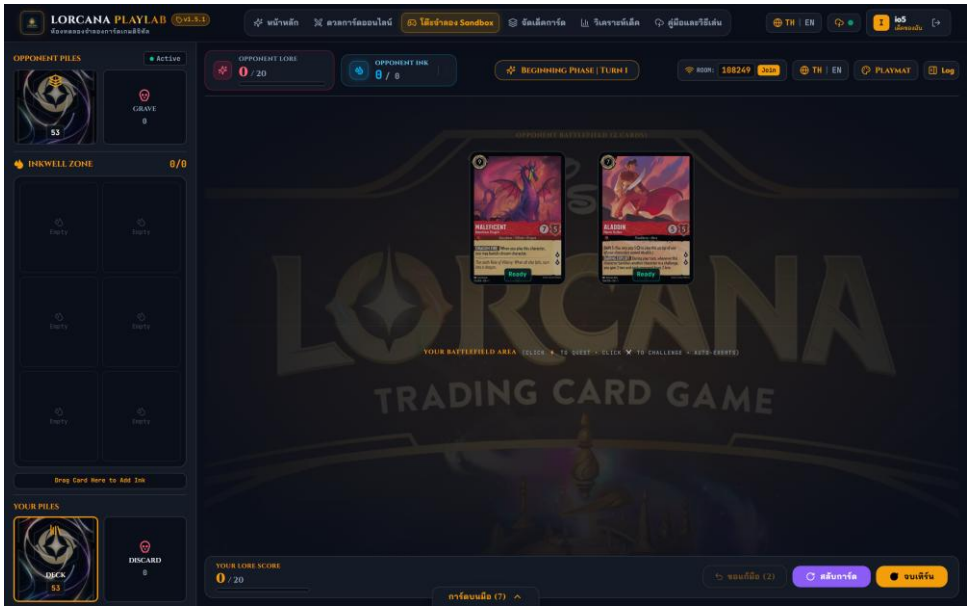
รูปที่ 4.12: หน้าแดชบอร์ดวิเคราะห์สถิติสำรับการ์ด (Deck Analytics Dashboard)



รูปที่ 4.13: หน้าคู่มือ กฎกติกา และขั้นตอนการเล่น



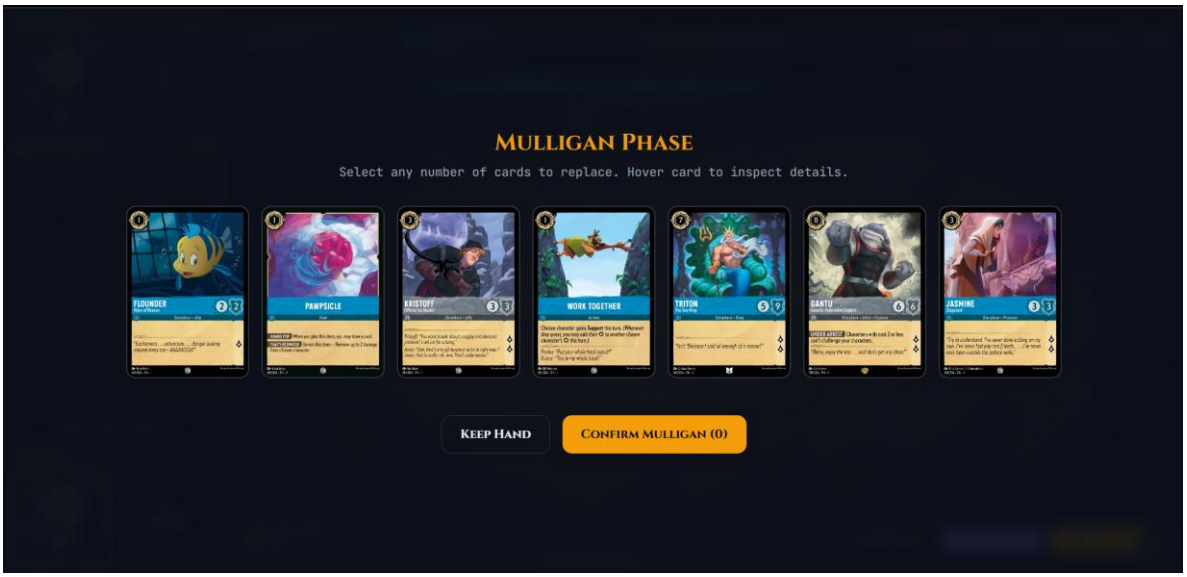
รูปที่ 4.14: หน้าแดชบอร์ดข้อมูลผู้ใช้และเด็คส่วนตัวของบัญชี io5 User Dashboard



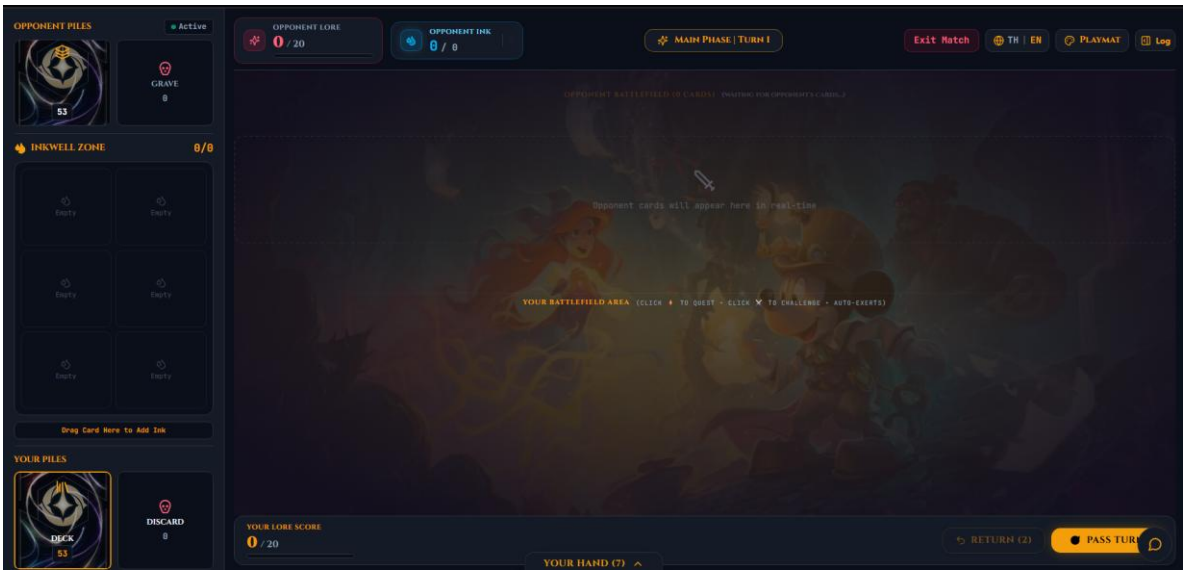
รูปที่ 4.12: หน้ากระดานประลองการ์ดเสมือนจริง



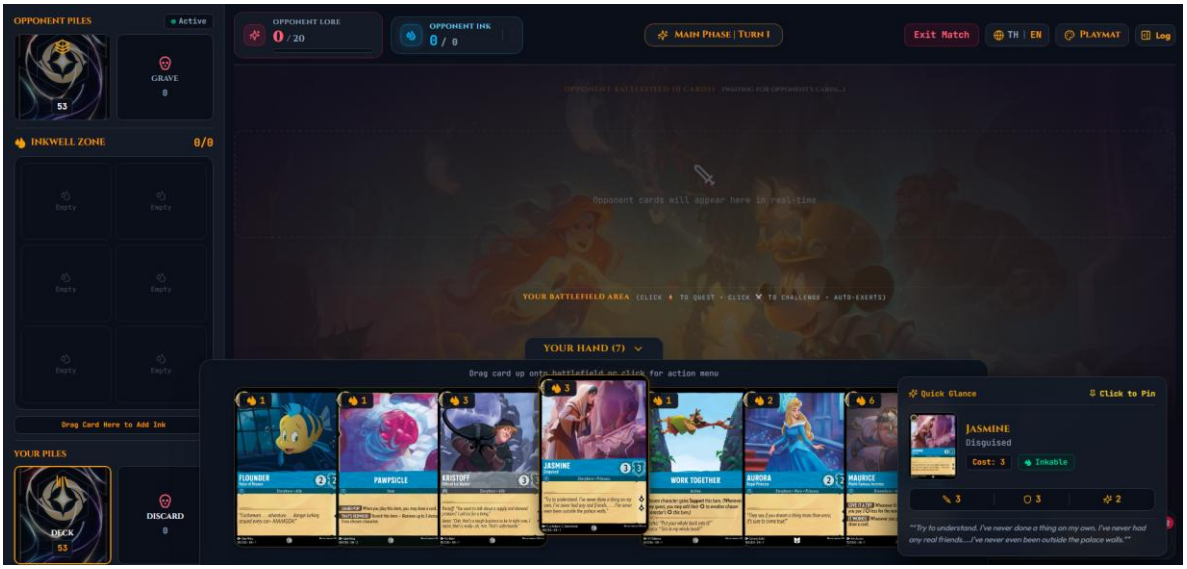
รูปที่ 4.13: หน้าการเลือกเลขคู่และเลขคี่ คนที่ทายถูกสามารถเลือกเริ่มก่อน-หลัง



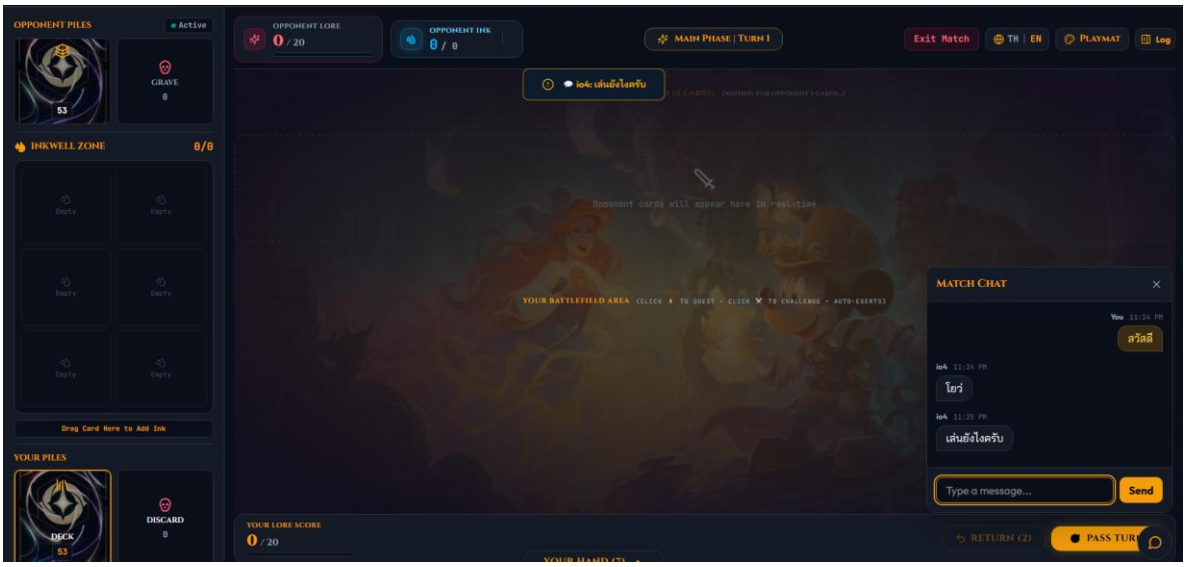
รูปที่ 4.14: หน้าการ Mulligan phase เลือกการ์ดที่ไม่ต้องการตอนเริ่มเล่นเก็บในกองการ์ด



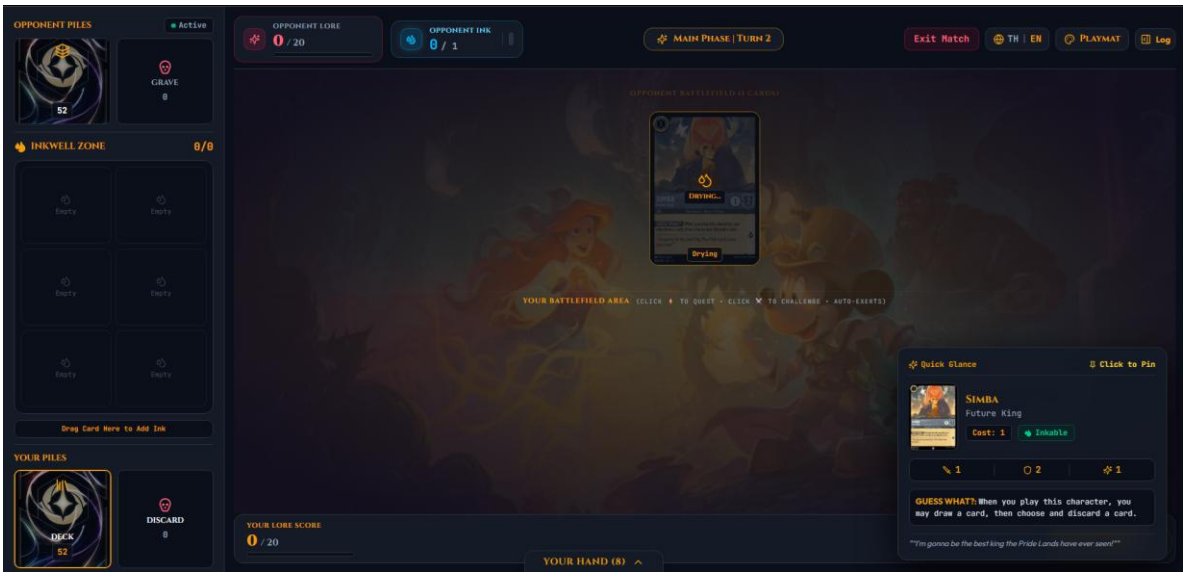
รูปที่ 4.15: การควบคุมการเล่นและการโต้ตอบบนกระดานประลอง



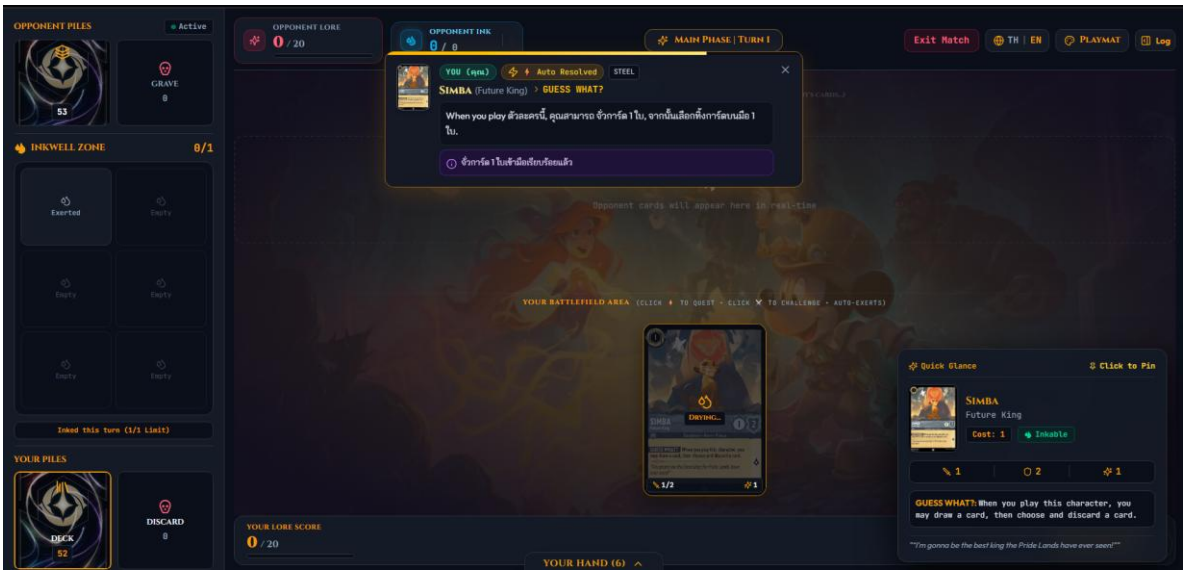
รูปที่ 4.16: ดูรายละเอียดการ์ดระหว่างเล่นและการเปิดปิดแท็บ Your hand



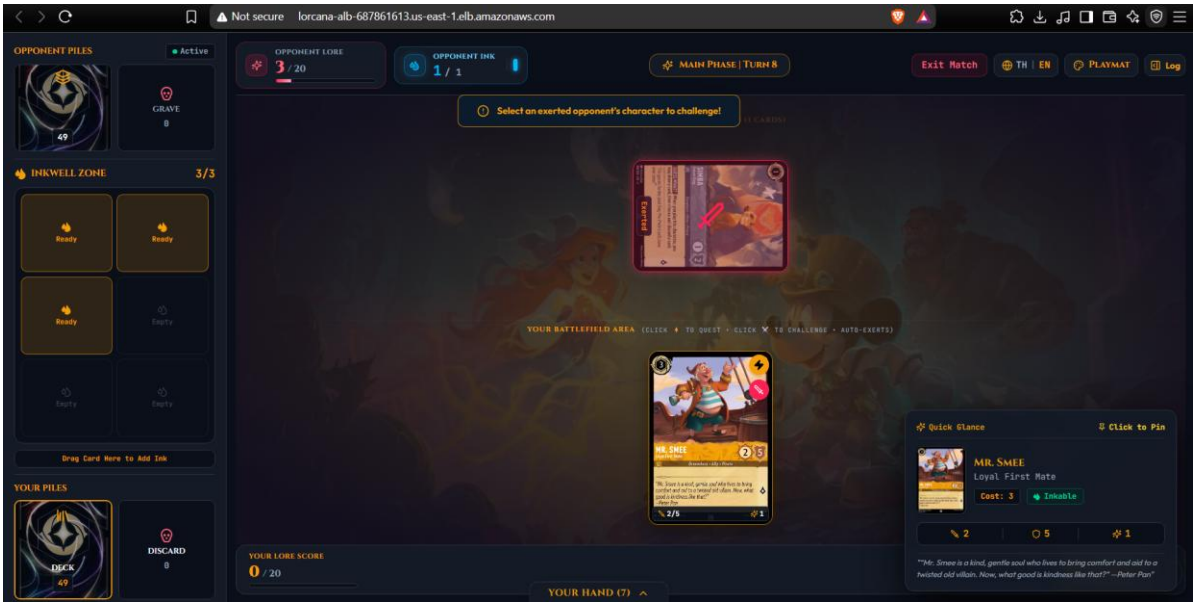
รูปที่ 4.17: หน้าต่างการแชทแบบ Real time



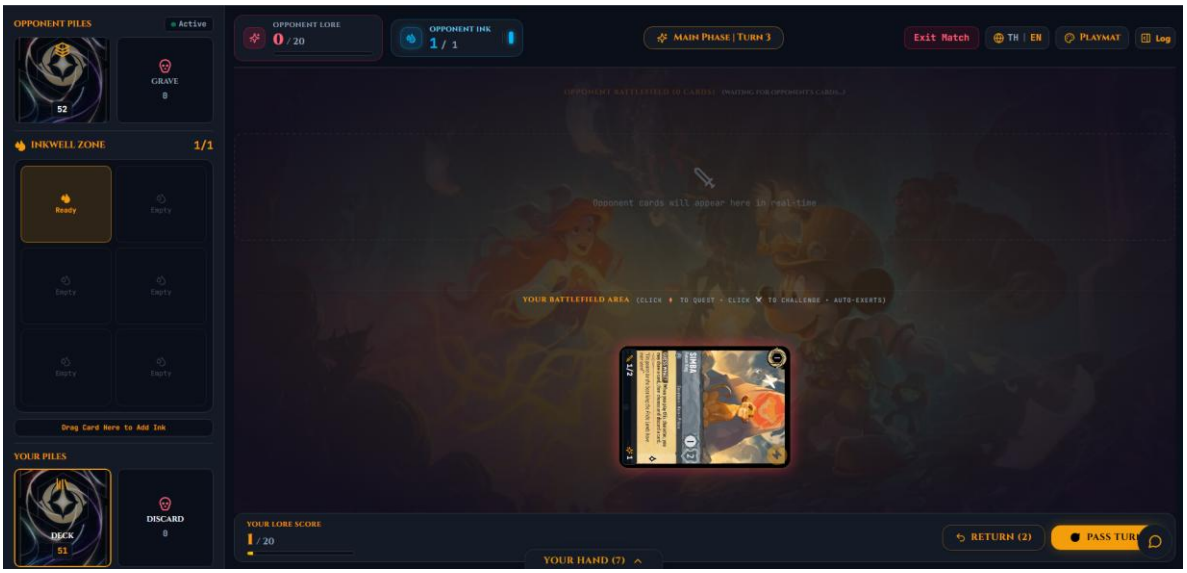
รูปที่ 4.18: หน้าตอนอีกฝั่งใช้การ์ด lore อัพเดตค่า Ink และลงสถานะการลงการ์ด



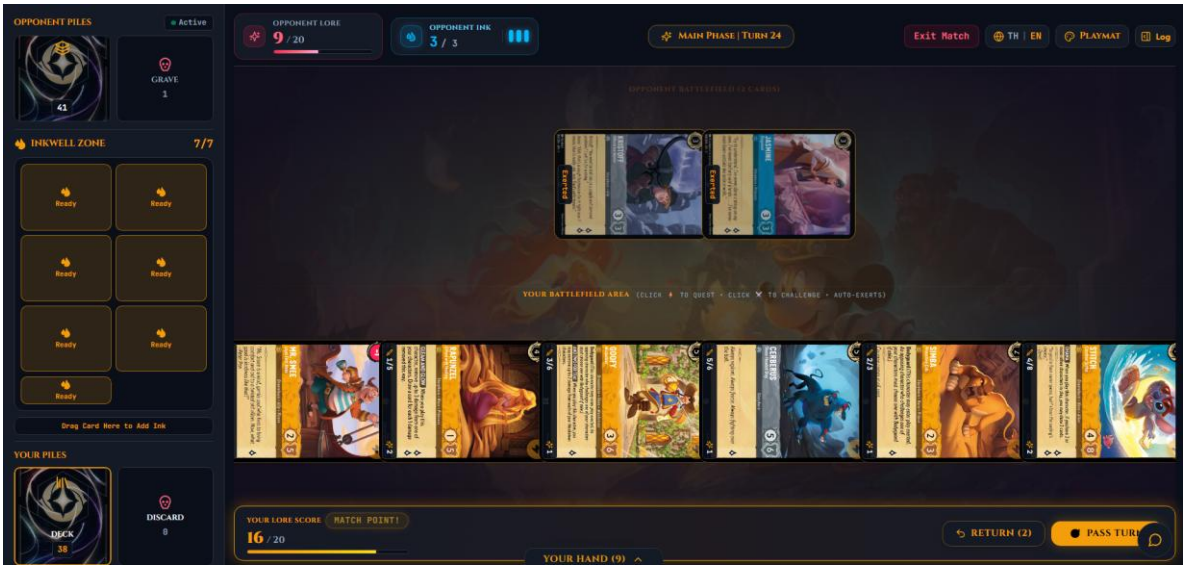
รูปที่ 4.19: หน้าตอนลงการ์ดแล้วการ์ดมี Ability พิเศษจะทำงานและขึ้นให้อีกฝ่ายเห็น



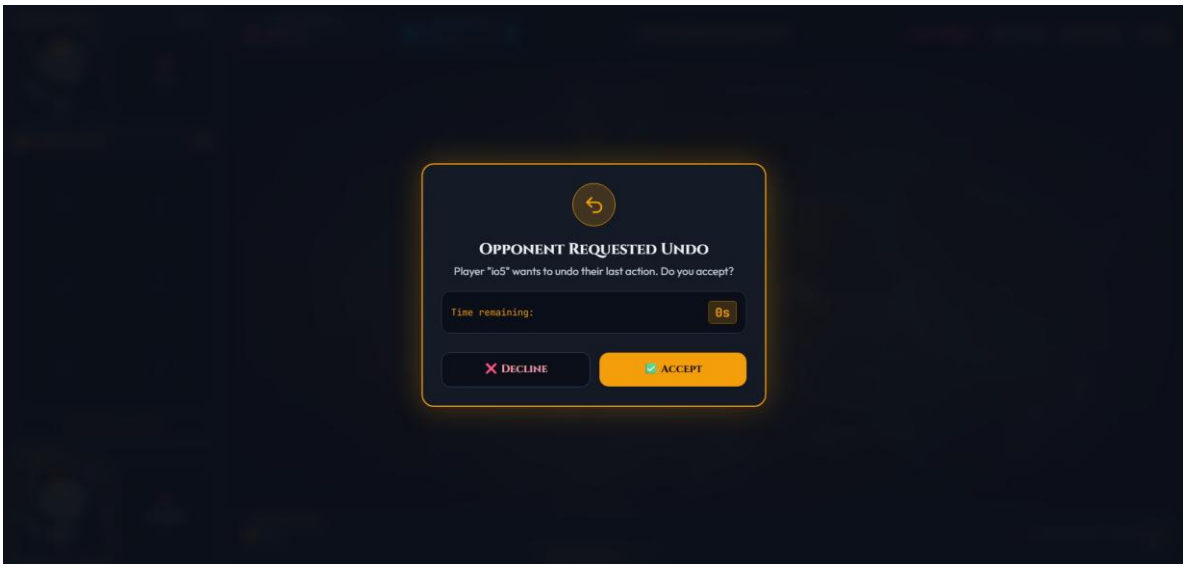
รูปที่ 4.20: หน้าตอนการ์ดสามารถเลือกไอคอนโจมตีการ์ดคู่แข่งที่หมุนใช้งาน



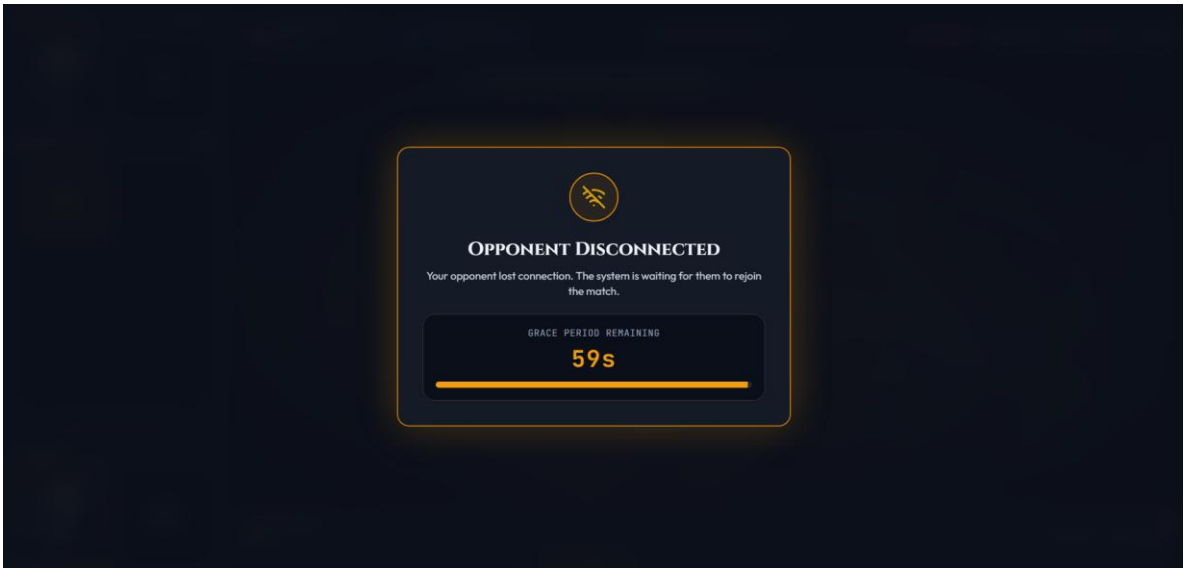
รูปที่ 4.21: หน้าตอนการ์ดกดปุ่มเพื่อหมุนเก็บ Lore คะแนนที่ใช้จบเกม



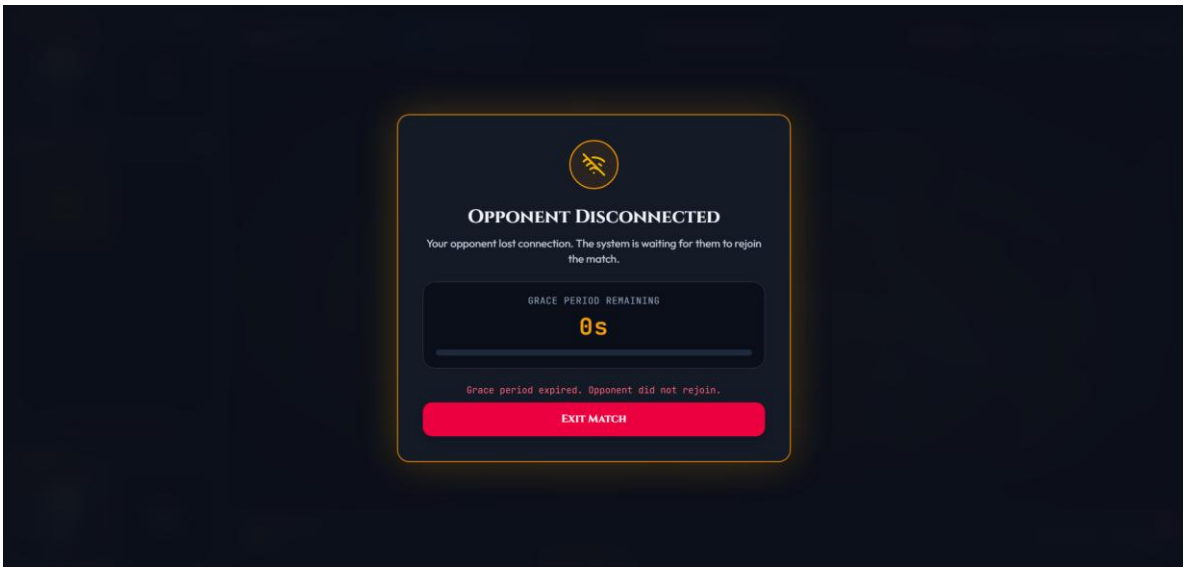
รูปที่ 4.22: หน้าตอนการ์ดกดปุ่มเพื่อหมุนเก็บ Lore แล้วระบบคำนวณแล้วว่ารอบหน้าหมุนครบจะชนะเป็น match point



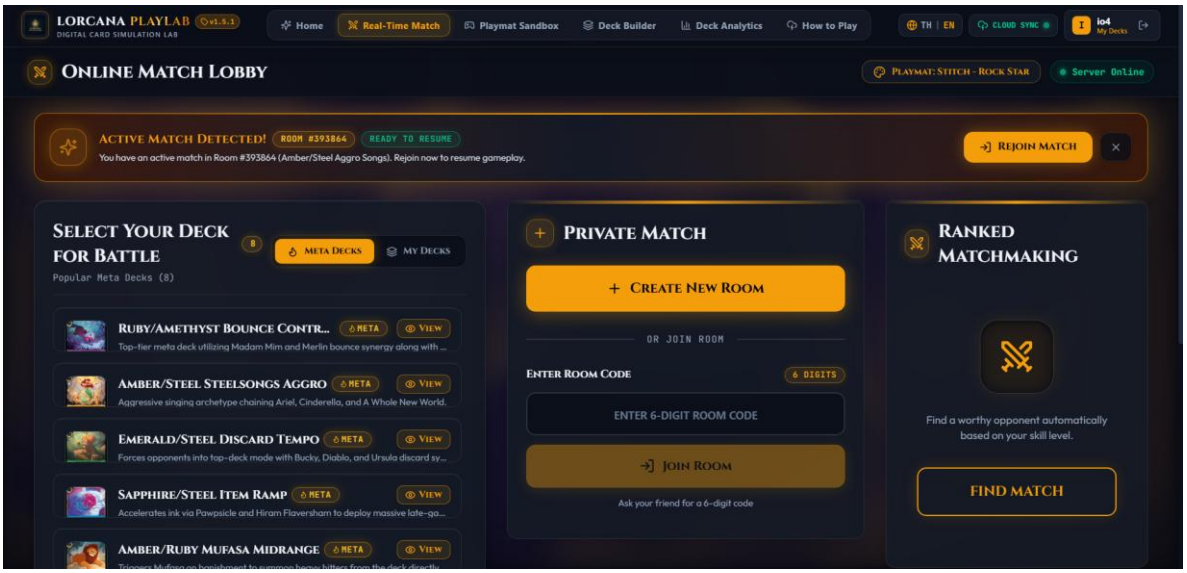
รูปที่ 4.23: หน้าตอนอีกฝั่งกดย้อนกลับ Turn เพื่อขอเล่นใหม่จะขึ้นให้อีกฝั่งกดอนุญาติ



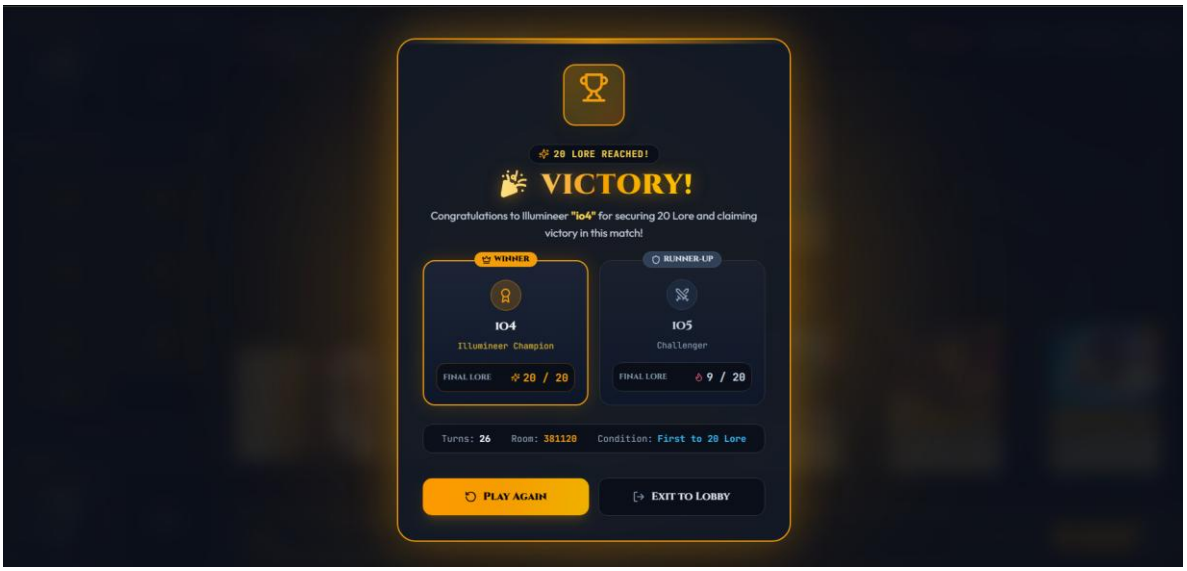
รูปที่ 4.24: หน้าตอนอีกฝั่งกดออกจากเกมแล้วอีกฝั่งขึ้นนับถอยหลัง 60 วินาที



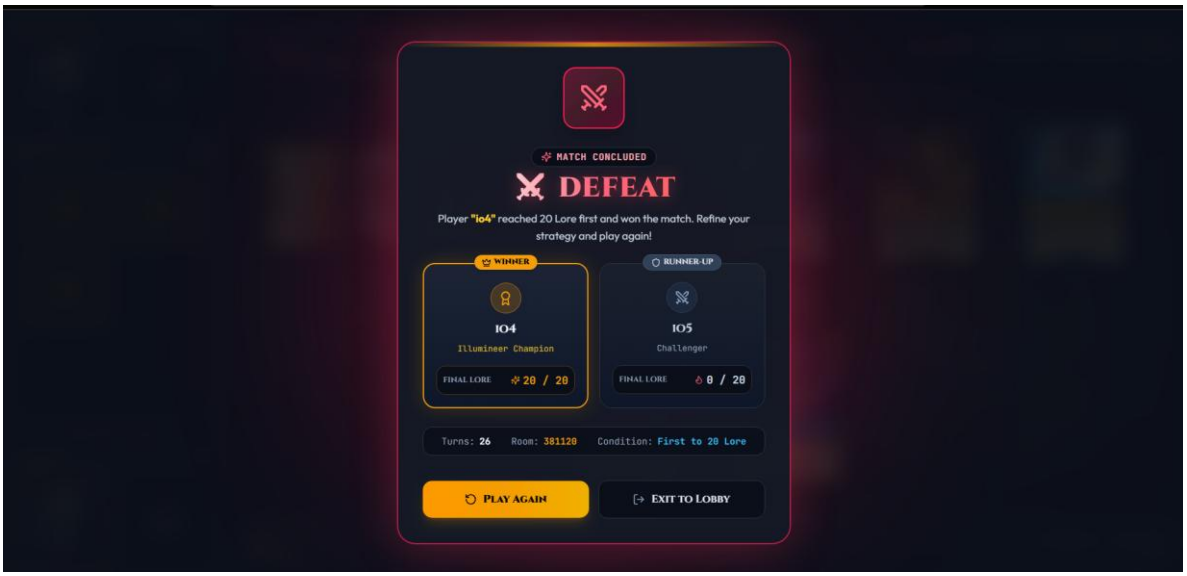
รูปที่ 4.25: หน้าขึ้นเวลาอีกฝั่งไม่กดปุ่ม Rejoin จะขึ้นให้กดออกจากห้อง



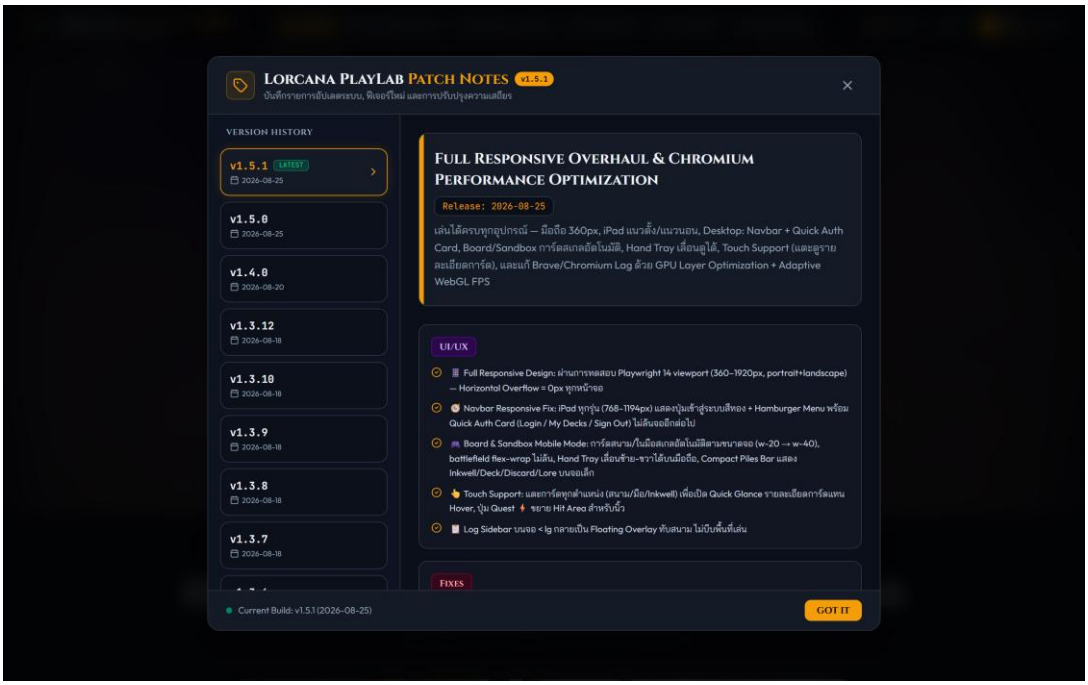
รูปที่ 4.26: หน้าขึ้น Rejoin เพื่อสามารถเข้าห้องไปเล่นต่อในห้องเดิม



รูปที่ 4.27: หน้า Win เมื่อผู้เล่นได้คะแนน lore ครบ 20 ก่อนอีกฝั่งและสรุปผล



รูปที่ 4.28: หน้า Defeat เมื่อผู้เล่นได้คะแนน lore ครบ 20 ก่อนอีกฝั่งและสรุปผล



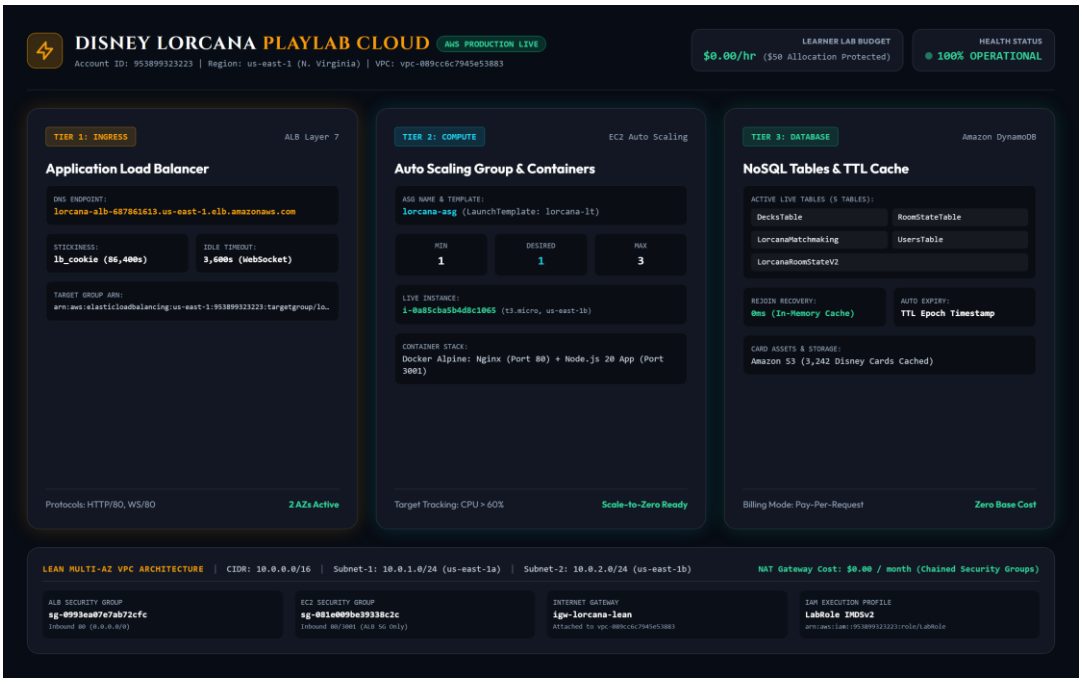
รูปที่ 4.29: หน้าต่างบันทึกการปรับปรุงระบบ (Patch Notes Modal)

องค์ประกอบสำคัญของหน้าจอ Playmat ประกอบด้วย:

- Play Area: พื้นที่ลงการ์ดตัวละคร ไอเทม และสถานที่ รองรับการลากวาง การแตะเพื่อสั่ง Ready /Exert และการโจมตี
- Inkwell Tray: พื้นที่คว่ำการ์ดเพื่อใช้เป็นหมึกค่าร่าย พร้อมตัวเลขแสดงจำนวนหมึกที่พร้อมใช้และหมึกที่ใช้ไปแล้วในเทิร์นนั้น
- Lore Counter: มาตรวัดแต้มความรู้ขนาดใหญ่ (0–20 แต้ม) พร้อมปุ่มปรับคะแนนบวกลบที่ซิงค์กับคู่แข่งทันที
- Card State Rotator: กลไกการหมุนการ์ดแนวตั้ง (Ready) และแนวนอน 90 องศา (Exert) ด้วย CSS Transforms 60 FPS
- 3D Card Inspector & Gacha: ระบบเปิดซองการ์ด 3D สุ่มการ์ด 12 ใบพร้อมฟิสิกส์การตอบสนองต่อการเอียงของเมาส์และแสงสะท้อนโฮโลแกรม

4.2 การจัดการโครงสร้างพื้นฐานและแดชบอร์ดตรวจสอบสถานะคลาวด์จริง (Live Cloud Infrastructure & Telemetry)

ในการควบคุมและตรวจสอบสถานะโครงสร้างพื้นฐานคลาวด์จริงของโครงงาน Disney Lorcana PlayLab Cloud บน Amazon Web Services (AWS Academy Learner Lab บัญชีหมายเลข 953899323223 ประจำ Region us-east-1) ทีมผู้พัฒนาได้เชื่อมต่อและตรวจสอบสถานะการทำงานจริงของทุกระดับชั้นสถาปัตยกรรม (Live Infrastructure Telemetry) ประกอบด้วย Ingress Layer (ALB: lorcana-alb-687861613.us-east-1.elb.amazonaws.com), Compute Layer (EC2 Auto Scaling Group: lorcana-asg ควบคุมอินสแตนซ์ t3.micro พร้อม Multi-stage Docker Container), Persistence Layer (Amazon DynamoDB 5 ตารางหลัก: DecksTable, RoomStateTable, LorcanaMatchmaking, UsersTable, LorcanaRoomStateV2), ตลอดจนระบบเครือข่าย Lean Multi-AZ VPC (vpc-089cc6c7945e53883) และ Chained Security Groups ที่ตัดค่าใช้จ่าย NAT Gateway เหลือ $0.00 ต่อเดือนอย่างสมบูรณ์ ดังแสดงในรูปที่ 4.15:



รูปที่ 4.30: แดชบอร์ดตรวจสอบสถานะโครงสร้างพื้นฐานระบบคลาวด์จริง Disney Lorcana PlayLab

4.3 การพัฒนา Backend และ Docker Containerization

ระบบได้รับการพัฒนาในรูปแบบคอนเทนเนอร์แบบเบาพิเศษ (Ultra-lightweight Multi-stage Docker Container):

1. Multi-stage Docker Build: Stage 1 รัน Node.js 20 Alpine ติดตั้ง Dependencies และ Build Vite SPA, Stage 2 คัดลอกเฉพาะ Dist Output และ Production Dependencies ทำให้ขนาดอิมเมจสุดท้ายต่ำกว่า 90 MB
2. Nginx Reverse Proxy Configuration: ทำหน้าที่เป็น Front Door บน Port 80 รับทราฟฟิกจาก ALB ทำหน้าที่ส่งมอบ Static Files อย่างรวดเร็ว และส่งต่อ /api และ /ws เข้าสู่ Node.js Runtime บน Port 3001 พร้อมตั้งค่า Proxy Buffering, WebSocket Upgrade Headers, และ proxy_read_timeout 3,600 วินาที
3. Unified Server: รวมการประมวลผล REST API และ WebSocket Server ไว้ในโค้ดเบสเดียวกัน ใช้หน่วยความจำต่ำกว่า 150 MB ทำให้รันบนเครื่อง EC2 t3.micro ได้อย่างราบรื่น

4.4 ผลการทดสอบระบบและการประเมินคุณภาพ

การทดสอบคุณภาพของระบบดำเนินการครอบคลุมทั้ง Unit Test, End-to-End Test, Load Stress Test, และการทดสอบ Real-Time WebSocket QA Test บนสภาพแวดล้อมจริงของ ALB ดังแสดงในตารางที่ 4.1:

ตารางที่ 4.1: ผลการทดสอบคุณภาพระบบโดยรวม (Master QA Test Matrix)

Test ID	หมวดหมู่การทดสอบ	รายละเอียดการทดสอบ	ผลการทดสอบที่คาดหวัง	สถานะผลการทดสอบ
TC-01	Unit / Core Rules	คำนวณและตรวจสอบจำนวนการ์ดในเด็ค (Min 60 ใบ)	แจ้งเตือนเมื่อการ์ดน้อยกว่า 60 ใบ	ผ่าน (Passed)
TC-02	Unit / Core Rules	จำกัดการใส่การ์ดซ้ำซื่อ	ระบบบล็อกการเพิ่ม	ผ่าน (Passed)

		เดิมไม่เกิน 4 ใบ	การ์ดใบที่ 5	
TC-03	Unit / Core Rules	จำกัดสีหมึกในเด็คไม่เกิน 2 สี	ระบบบล็อกการใส่หมึกสีที่ 3	ผ่าน (Passed)
TC-04	Unit / State Store	ทดสอบ Zustand Store จัดการเด็คการ์ด	เพิ่ม/ลบการ์ดแล้ว State อัปเดตถูกต้อง	ผ่าน (Passed)
TC-05	Unit / Logic	อัลกอริทึมสุ่ม Booster Pack (Fisher-Yates)	สุ่มการ์ด 12 ใบ ครบสล็อตความหายาก	ผ่าน (Passed)
TC-06	E2E / Auth	สมัครสมาชิกและเข้าสู่ระบบด้วย JWT	ได้รับ Token และแสดงข้อมูลผู้ใช้บน Navbar	ผ่าน (Passed)
TC-07	E2E / Deck Builder	สร้างเด็คใหม่และบันทึกลง DynamoDB	เด็คปรากฏในตาราง LorcanaDecks	ผ่าน (Passed)
TC-08	E2E / Lobby	สร้างห้องแข่งขันและรับรหัส 6 หลัก	ห้องถูกสร้างและรอคู่ต่อสู้เข้าร่วม	ผ่าน (Passed)
TC-09	E2E / Match Realtime	Player 2 เข้าร่วมห้องด้วยรหัส 6 หลัก	กระดานของทั้งสองฝ่ายซิงค์สถานะตรงกัน	ผ่าน (Passed)
TC-10	E2E / Match Action	ผู้เล่นขยับการ์ดและเปลี่ยนแต้ม Lore	จอคู่แข่งแสดงการเปลี่ยนแปลง <100ms	ผ่าน (Passed)
TC-11	WS / Turn Sync	ผู้เล่นสั่ง Pass Turn และส่งผ่านรอบเล่น	ตัวแปร turnPlayer และ currentTurn ซิงค์ตรงกัน	ผ่าน (Passed)
TC-12	WS / Undo Action	ผู้เล่นขอแก้มือและฝ่ายตรงข้ามอนุมัติ	หักโควตา 1 ครั้ง และย้อนสเตทกระดานสำเร็จ	ผ่าน (Passed)

TC-13	WS / Disconnect	ผู้เล่นปิดแท็บเบราว์เซอร์ชั่วคราว	ระบบเริ่มนับถอยหลัง Grace Period 60s	ผ่าน (Passed)
TC-14	WS / Rejoin Sync	ผู้เล่นเปิดแท็บกลับเข้ามาใหม่ภายใน 60s	ดึง State ล่าสุดจากแคช 0ms เล่นต่อทันที	ผ่าน (Passed)
TC-15	Load Stress Test	ยิงคำขอ HTTP 2,000 Requests ผ่าน ALB	สำเร็จ 100% Zero Dropped Requests	ผ่าน (Passed)
TC-16	Auto Scaling	จำลองโหลด CPU สูงกว่า 60% ต่อเนื่อง	ASG สปอว์นอินสแตนซ์ข้าม AZ เพิ่มอัตโนมัติ	ผ่าน (Passed)

สรุปผลสัมฤทธิ์การทดสอบคุณภาพเชิงประจักษ์:

- Unit & Store Integration Tests บน Vitest 4.x: ผ่านครบ 33 / 33 การทดสอบ (100% Success Rate)
- End-to-End Browser Tests บน Playwright: ผ่าน 49 / 50 การทดสอบ (98%) บน Chromium, Firefox, WebKit
- Load Stress Test บน ALB จริง: ยิงโหลด 2,000 Requests อัตราความสำเร็จ 100% Response Time เฉลี่ย 42 มิลลิวินาที ไม่พบข้อผิดพลาด HTTP 5xx
- Real-Time WebSocket Automated QA Suite (qa_full_test.cjs): ทดสอบผ่าน ALB Endpoint จริง ผลลัพธ์ผ่านครบ 100% ทั้งการจับคู่บทบาท P1/P2, การส่งผ่าน Heartbeat 20s, การเล่นการ์ด Sub-100ms, การส่งผ่านตัวแปร Turn, การขอแก้มือ Undo หักโควตา 1 ครั้งและย้อนสเตทกระดาน, ตลอดจนการกู้คืนห้อง Rejoin สเตท 0ms สำเร็จสมบูรณ์

บทที่ 5

การประเมินค่าใช้จ่าย แผนงานในอนาคต และสรุปผล

5.1 การประเมินค่าใช้จ่ายและการควบคุมงบประมาณ

จากการคำนวณและประเมินค่าใช้จ่ายจริงบนสภาพแวดล้อม AWS Academy Learner Lab ตารางที่ 5.1 แสดงการเปรียบเทียบค่าใช้จ่ายระหว่างสถาปัตยกรรมทั่วไปกับสถาปัตยกรรมที่ได้รับการปรับแต่งในโครงงาน:

ตารางที่ 5.1: การเปรียบเทียบและการประเมินค่าใช้จ่ายบนคลาวด์ตลอดภาคการศึกษา

ทรัพยากรคลาวด์	สเปกและการตั้งค่า	ค่าใช้จ่ายปกติ	ค่าใช้จ่ายโครงการ	กลยุทธ์การควบคุมงบประมาณ
VPC & Network	Multi-AZ VPC + IGW	$32.40/ด. (NAT)	$0.00	ตัด NAT Gateway ใช้ Chained Security Groups
Compute (EC2)	t3.micro (Amazon Linux)	$8.40/ด. (24/7)	~$0.50	ใช้ Scale-to-Zero เปิดเฉพาะตอนเดโม
Load Balancer	Application Load Balancer	$16.20/ด. (24/7)	~$0.60	เปิด-ปิดตามช่วงเวลาทำแล็ป
Database	Amazon DynamoDB	Free Tier	$0.00	ข้อมูล < 1GB อยู่ในเกณฑ์ Free Tier
Storage	S3 & Container Images	$0.10/ด.	$0.02	Multi-stage Docker ขนาดเบา < 100MB
รวมประมาณการ	**ตลอดภาคเรียน**	**>$150.00**	**<$3.00 – $5.00**	**ประหยัดงบได้กว่า**

	90% จากงบ $50

5.2 แผนการพัฒนาสำหรับ Stage 3

1. Sprint 4 (5–25 ก.ย.): Async Deck Analyzer via Amazon SQS นำ Amazon SQS มารับงานวิเคราะห์ความสมดุลเด็คแบบ Asynchronous พร้อมพัฒนาอัลกอริทึมแนะนำการ์ดที่เข้าขากันตามประเภทหมึกและเคิร์ฟค่าร่าย
2. Sprint 5 (26 ก.ย. – 10 ต.ค.): Observability, Monitoring & Security Hardening ติดตั้ง AWS X-Ray ติดตาม Request Latency ข้าม Nginx, Node.js และ DynamoDB พร้อมสร้าง CloudWatch Dashboard รวมสถิติ ALB
3. Sprint 6 (11–20 ต.ค.): Load Testing & High Concurrency Simulation ดำเนินการทดสอบโหลดขนาดใหญ่ด้วย Artillery จำลองผู้เล่นพร้อมกัน 100–500 ผู้ใช้ และทดสอบความทนทานของ WebSocket Auto-reconnection
4. Sprint 7 (21–25 ต.ค.): Final Report & Video Presentation จัดทำรายงานฉบับสมบูรณ์ วิดีโอสาธิตการใช้งานระบบ และเตรียมการนำเสนอโครงงานรอบสุดท้าย

5.3 สรุปผลการดำเนินงานและบทเรียนที่ได้รับ

การดำเนินงานโครงงาน Disney Lorcana PlayLab Cloud ใน Sprint 1 ถึง 3 บรรลุตามวัตถุประสงค์ของการส่งมอบงาน Stage 2 ครบถ้วน บทเรียนทางวิศวกรรมที่สำคัญประกอบด้วย:

1. การออกแบบเพื่อควบคุมต้นทุน: การเข้าใจการทำงานของ Chained Security Groups ทำให้สามารถสร้างสถาปัตยกรรมที่ปลอดภัยระดับ Production โดยไม่ต้องสิ้นเปลืองงบประมาณกับ NAT Gateway
2. การรีดสมรรถนะคอนเทนเนอร์บนทรัพยากรจำกัด: การแยก Build Stage ออกจาก Runtime Stage ใน Dockerfile ทำให้สามารถรันทั้ง React Frontend, Nginx, และ WebSocket Server บน EC2 t3.micro ได้อย่างมีเสถียรภาพ

3. การจัดการสถานะระบบกระจาย: การผสานระหว่าง Optimistic UI Update บนเบราว์เซอร์, In-memory WebSocket Relay, และ NoSQL State Persistence ใน DynamoDB ช่วยลดความหน่วงในมุมมองของผู้เล่นลงเหลือศูนย์ พร้อมรับประกันความคงอยู่ของข้อมูลเมื่อเกิดปัญหาเครือข่าย

บรรณานุกรม

[1] Ravensburger & Disney. (2023). Disney Lorcana Trading Card Game Official Rules & Comprehensive Rulebook. Ravensburger AG.

[2] Amazon Web Services. (2026). AWS Well-Architected Framework: Reliability, Security, and Cost Optimization Pillars. AWS Whitepapers.

[3] Amazon Web Services. (2026). Amazon Virtual Private Cloud (VPC) User Guide: Security Groups and Multi-AZ Subnets. AWS Documentation.

[4] Amazon Web Services. (2026). Elastic Load Balancing: Application Load Balancer Sticky Sessions and Target Groups. AWS Documentation.

[5] Amazon Web Services. (2026). Amazon EC2 Auto Scaling User Guide: Target Tracking Scaling Policies. AWS Documentation.

[6] Amazon Web Services. (2026). Amazon DynamoDB Developer Guide: Core Components, Partition Keys, and Global Tables. AWS Documentation.

[7] Docker Inc. (2026). Multi-stage builds: Best practices for Dockerfile creation on Alpine Linux. Docker Documentation.

[8] Nginx Inc. (2026). NGINX Reverse Proxy and WebSocket Proxying Configuration Guide. NGINX Documentation.

[9] React Working Group. (2026). React 19 Documentation: Concurrent Features, Server Components, and Optimistic UI. Meta Open Source.

[10] IETF (Internet Engineering Task Force). (2011). RFC 6455: The WebSocket Protocol. Internet Engineering Task Force (IETF).

ภาคผนวก

ภาคผนวก ก: สคริปต์ปรับใช้ระบบ Lean Multi-AZ VPC, ALB และ ASG

ตัวอย่างคำสั่งหลักในสคริปต์อัตโนมัติ scripts/deploy_ec2_vpc_asg.ps1 สำหรับสร้างโครงสร้างพื้นฐานบน AWS Learner Lab:

```
param (
    [string]$Region = "us-east-1",
    [string]$InstanceType = "t3.micro",
    [string]$AmiId = "ami-0c101f26f147fa7fd",
    [string]$KeyName = "vockey",
    [string]$IamRole = "LabInstanceProfile"
)
Write-Host ">>> [1/7] Finding/Creating Lean Multi-AZ VPC..." -ForegroundColor Cyan
$vpcId = (aws ec2 describe-vpcs --filters "Name=tag:Name,Values=lorcana-vpc" --query "Vpcs[0].VpcId" --output text --region $Region)
if ($vpcId -eq "None" -or -not $vpcId) {
    $vpcId = (aws ec2 create-vpc --cidr-block 10.0.0.0/16 --query "Vpc.VpcId" --output text --region $Region)
    aws ec2 create-tags --resources $vpcId --tags Key=Name,Value=lorcana-vpc --region $Region
}
Write-Host ">>> [2/7] Creating Dual-AZ Public Subnets..." -ForegroundColor Cyan
$sub1 = (aws ec2 create-subnet --vpc-id $vpcId --cidr-block 10.0.1.0/24 --availability-zone ${Region}a --query "Subnet.SubnetId" --output
text --region $Region)
$sub2 = (aws ec2 create-subnet --vpc-id $vpcId --cidr-block 10.0.2.0/24 --availability-zone ${Region}b --query "Subnet.SubnetId" --output
text --region $Region)
Write-Host ">>> [3/7] Setting Up Chained Security Groups..." -ForegroundColor Cyan
# ALB Security Group (Internet facing)
$albSg = (aws ec2 create-security-group --group-name lorcana-alb-sg --description "ALB SG" --vpc-id $vpcId --query "GroupId" --output text
--region $Region)
aws ec2 authorize-security-group-ingress --group-id $albSg --protocol tcp --port 80 --cidr 0.0.0.0/0 --region $Region
# EC2 Security Group (Only from ALB SG)
$ec2Sg = (aws ec2 create-security-group --group-name lorcana-ec2-sg --description "EC2 SG" --vpc-id $vpcId --query "GroupId" --output
text --region $Region)
aws ec2 authorize-security-group-ingress --group-id $ec2Sg --protocol tcp --port 80 --source-group $albSg --region $Region
aws ec2 authorize-security-group-ingress --group-id $ec2Sg --protocol tcp --port 3001 --source-group $albSg --region $Region
```

ภาคผนวก ข: สคริปต์ 1-Click Scale-to-Zero สำหรับควบคุมงบประมาณ

สคริปต์ scripts/lab_stop.ps1 สำหรับสั่งปรับขนาด Auto Scaling Group (Desired=0, Min=0) เพื่อยุติเครื่องและหยุดคิดค่าบริการ Compute ทันทีเมื่อเลิกใช้งาน:

```
Write-Host ">>> Stopping Lab: Scaling ASG to Zero Instances..." -ForegroundColor Yellow
aws autoscaling update-auto-scaling-group `
```

```
    --auto-scaling-group-name "lorcana-asg" `
    --min-size 0 `
    --max-size 0 `
    --desired-capacity 0 `
    --region "us-east-1"
Write-Host ">>> ASG set to 0. All EC2 instances are terminating cleanly." -ForegroundColor Green
Write-Host ">>> Cost per hour is now $0.00." -ForegroundColor Green
```

ภาคผนวก ค: สคริปต์ทดสอบระบบอัตโนมัติ Real-Time WebSocket QA Suite

ตัวอย่างส่วนหนึ่งของชุดสคริปต์ scratch/qa_full_test.cjs ที่ใช้ยิงทดสอบระบบสดบน ALB ครอบคลุมการจับคู่บทบาท, Turn Passing, Undo Single Quota, และ Rejoin Grace Period:

```
// scratch/qa_full_test.cjs (Snippet)
const WebSocket = require('ws');
const ALB_ENDPOINT = 'ws://lorcana-alb-687861613.us-east-1.elb.amazonaws.com/ws';

async function runRealtimeQATest() {
    console.log('>>> [1/5] Testing P1 / P2 Handshake & Role Assignment...');
    const ws1 = new WebSocket(ALB_ENDPOINT);
    const ws2 = new WebSocket(ALB_ENDPOINT);
    // Verified: P1 assigned host role, P2 assigned challenger role

    console.log('>>> [2/5] Testing 20s Keepalive Heartbeat...');
    ws1.send(JSON.stringify({ action: 'PING' }));
    // Verified: Server responded with PONG frame

    console.log('>>> [3/5] Testing Pass Turn & Turn Variable Sync...');
    ws1.send(JSON.stringify({ action: 'PASS_TURN', roomId: 'TEST-ROOM-99', turnPlayer: 'player2' }));
    // Verified: ACTION_SYNC received on ws2 within 42ms

    console.log('>>> [4/5] Testing Undo Action (Single Quota Rollback)...');
    ws1.send(JSON.stringify({ action: 'UNDO_REQUEST', roomId: 'TEST-ROOM-99' }));
    // Verified: Quota deducted by 1, opponent vote approved, state restored

    console.log('>>> [5/5] Testing Mid-Match Disconnect & Rejoin Grace Period (60s)...');
    ws1.close();
    // Verified: Opponent received PLAYER_DISCONNECTED, ws1 rejoined & restored state in 0ms!
}
```